

A Primer: ARM[®] Trace

Including:

ETM[™], ETB and Serial Wire Viewer, JTAG and SWD



V 2.1



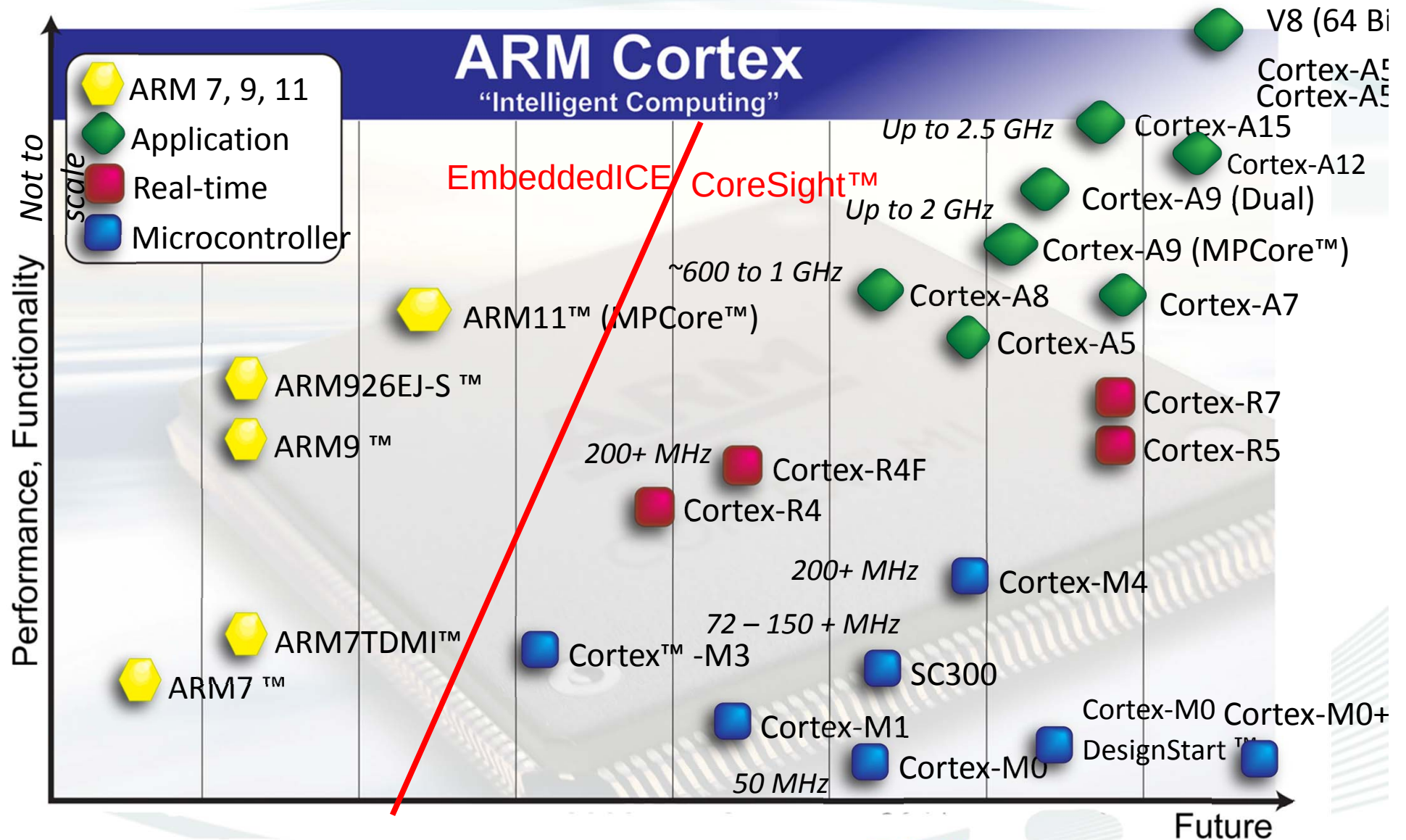
Agenda

- ▶ Introduction
- ▶ How we talk to your CPU using JTAG or SWD.
- ▶ Trace. ETM™, ETB and SWV. How are they different ?
- ▶ Triggers, filters and stuff.....
- ▶ Instrumentation. DCC and ITM.
- ▶ Breakpoints and Watchpoints
- ▶ What is trace good for ? Some points.

Debugging Skill Set

- ▶ Trace: around for a long, long time.
- ▶ Not many use it – maybe < 5%. The best do.
- ▶ Trace helps you find the nastiest problems...
- ▶ And often fairly easily... just like in...Mexico
- ▶ ...is good for testing software and hardware too.
- ▶ Developer productivity is very important....
- ▶ So – let's look at the various technologies ARM has....

ARM Public Processor Roadmap

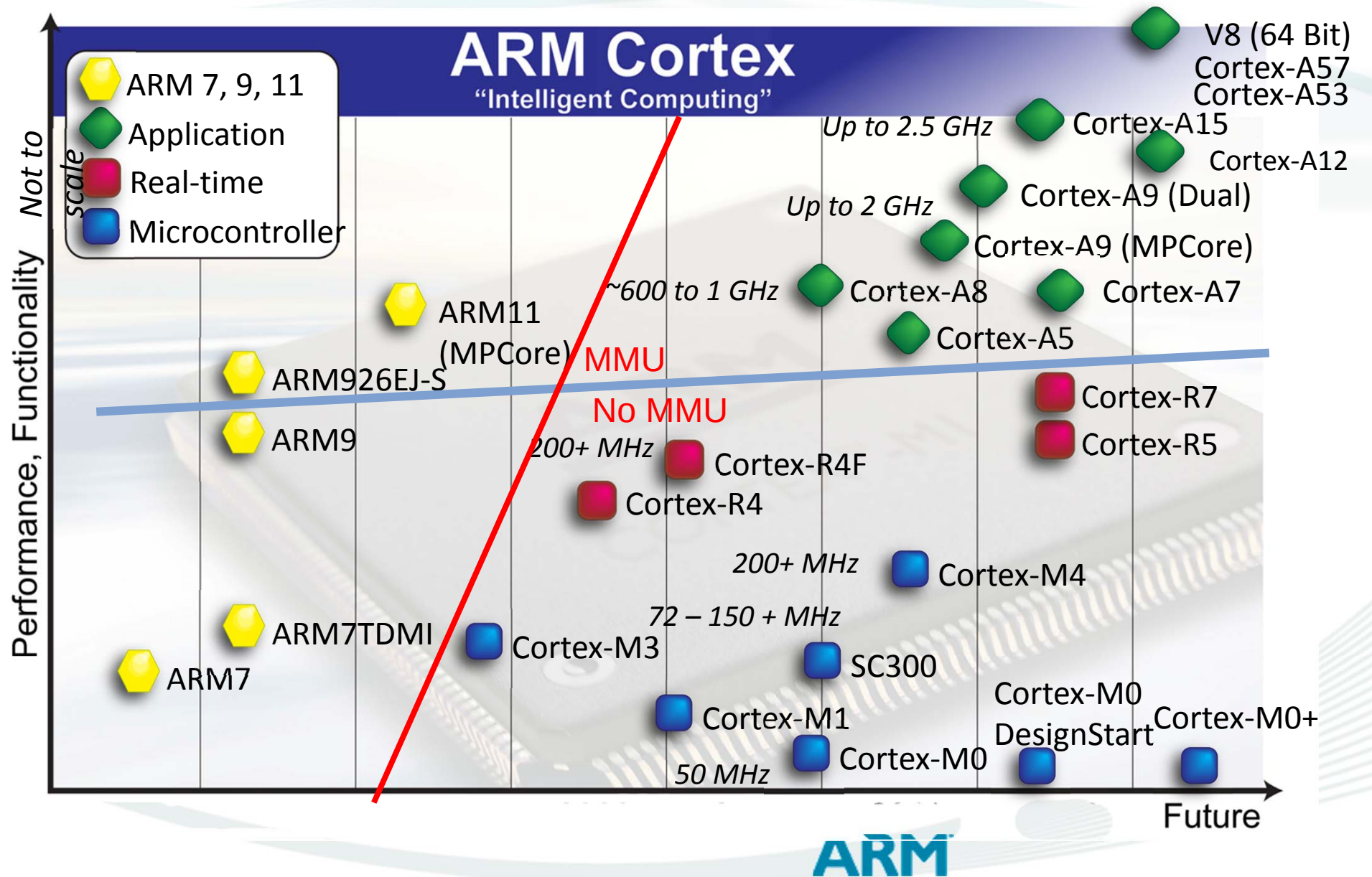


All dates are

2007

ARM

ARM Public Processor Roadmap



Versions, cores and architectures ?

- ▶ What is the difference between ARM7™ and ARMv7 ?
- ▶ Search for ARM architecture on Wikipedia to get the full list.
- ▶ ARM doesn't make chips....well maybe a few test chips.

Family Architecture Cores

ARM7TDMI	ARMv4T	ARM7TDMI(S)
ARM9 ARM9E	ARMv5TE(J)	ARM926EJ-S, ARM966E-S
ARM11	ARMv6 (T2)	ARM1136(F), 1156T2(F)-S, 1176JZ(F), ARM11 MPCore™
Cortex-A	ARMv7-A	Cortex A8, A9, A9MP
Cortex-R	ARMv7-R	Cortex R4(F)
Cortex-M	ARMv7-M	Cortex M3, M4
	ARMv6-M	Cortex M1, M0

What is Trace good for ? Part 1

- ▶ Tells where the program has been and how it got there.
- ▶ and for how long.
- ▶ Nasty problems can often be found quickly with trace.
- ▶ Especially where the bug occurs a long time before the consequences are seen.
- ▶ Or where the state of the system disappears with a change in scope(s). What caused the problem is off the stack.
- ▶ Race Conditions. A really big benefit here !
- ▶ Asynchronous events lining up.... as in an RTOS.

What is Trace good for ? Part 2

These are the types of problems that can be found with trace:

- ▶ Pointer problems.
- ▶ Illegal instructions and data aborts (such as misaligned writes).
- ▶ Code overwrites – writes to Flash, unexpected writes to peripheral registers (SFRs), corrupted stack.
- ▶ Out of bounds data. Uninitialized variables and arrays.
- ▶ Slow programs – is something eating up your CPU time ?
- ▶ No trace availability responsible for ongoing problems.

What is Trace good for ? Part 3

These are the types of problems that can be found with trace:

- ▶ Stack overflows. What causes the stack to grow bigger than it should ?
- ▶ **Runaway programs:** your program has gone off into the weeds and you need to know what instruction caused this. Is very tough to find these problems without a trace. But this is a very common problem. Stack not always has the answer.
- ▶ Communication protocol and timing issues.
- ▶ Profile Analyzer. Where is the CPU spending its time ?
- ▶ Code Coverage. Was all the code exercised ? Might be a certification requirement (FDA & FAA etc).
- ▶ Execution Profiling: times to execute source code.

Overall ARM Debugging Modules

▶ **EmbeddedICE:**

- The original modules for ARM processors.
- JTAG port.
- ETM, ETB, breakpoints, watchpoints.
- ARM7™, ARM9™, ARM11™

▶ **CoreSight : newer debug technology**

- Found mostly on Cortex processors.
- JTAG & SWD (Serial Wire Debug) ports.
- ETM, ETB, SWV, breakpoints, watchpoints.
- PTM (Program Trace Macrocell) Cortex-A9, Cortex-A15

▶ Some items are optional – *check your datasheet.*

What CoreSight delivers to you:

- ▶ Provides program flow and other activity from the processor.
- ▶ And in real-time without:
 - intrusion, stolen CPU cycles or instrumentation code.
 - And without stopping the CPU including most configuration !
- ▶ This means you can debug on-the-fly !
- ▶ Can see:
 - Program counters
 - Data reads and writes
 - Exception (including interrupt) activity...and all are timestamped !
 - CPU counters.

The debugging parts of ARM processors:

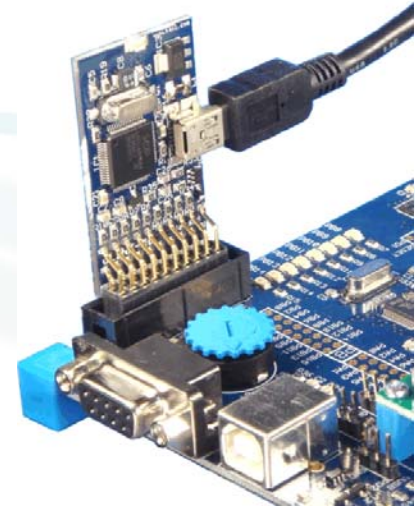
- ▶ **JTAG debugger:** (ICE & CoreSight)
 - JTAG is really the port to the debugging module in core.
- ▶ **SWD – Serial Wire Debug:** (CoreSight)
 - A 2 pin alternative to JTAG – same capabilities. (no boundary scan)
- ▶ **ETM:** Embedded Trace Macrocell (ICE & CoreSight)
 - Sends out PC and data reads and writes as fast as the core is. (E.ice)
 - CoreSight only sends out all PC values. Use SWV for other data.
- ▶ **PTM:** Like ETM but for Cortex-A9 and later.
- ▶ **ETB:** Embedded Trace Buffer (ICE & CoreSight)
 - Get ETM info out the JTAG connector – not ETM connector.
 - Is visible to the CPU.
- ▶ **SWV:** Serial Wire Viewer (CoreSight only)
 - 1 wire output: can see PC samples, data r/w and exceptions + more.
- ▶ **MEM-AP:** Memory Access Port - read/write memory real time.

A few more bits 'n pieces:

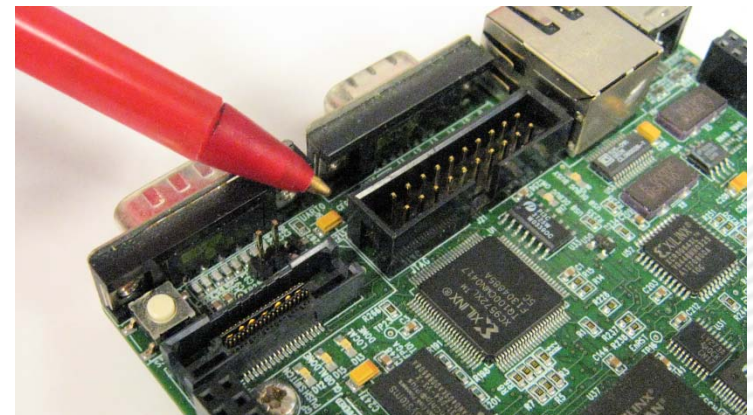
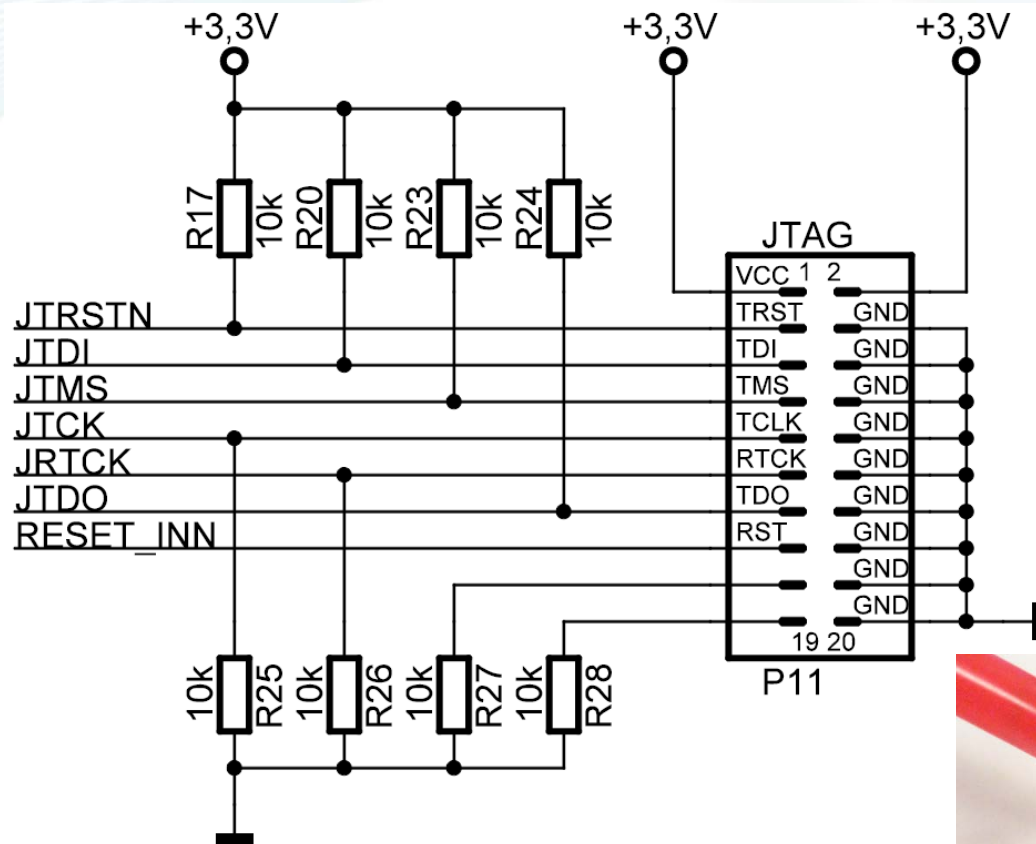
- ▶ These debug modules sit on the chip beside the core.
- ▶ Can see internal busses.
- ▶ Do not affect speed of core or steal cycles.
- ▶ We will call this “real-time”. Or Non-intrusive.
- ▶ Data Communication Channel. EmbeddedICE
- ▶ ITM[®] – Instrumentation Trace Macrocell.
- ▶ These are used to send messages or data out the JTAG or SWO ports in real-time. More on these two later...
- ▶ But ITM is really cool as you will see. Almost no code needed
- ▶ Multicore Debug – cross triggering between cores.

JTAG Debugging

- ▶ This is how we talk to the core.
- ▶ With this setup we can:
 - Start and stop program.
 - Single Step one source or assembler line.
 - Set breakpoints and watchpoints when CPU is stopped.
 - Note: with ARM CoreSight can set breakpoints on the fly
 - Read and write memory only when CPU is stopped. (CoreSight can !)
 - Program FLASH memory. Internal or External. Via a debugger.
 - JTAG used to test circuit board connections from chip to chip. (boundary scan)
- ▶ Can stop CPU then look and see state of system.
- ▶ Much better than monitor systems.
- ▶ Debug instructions are put directly into the core pipeline.
- ▶ JTAG adapters are called many things – but they are not in-circuit emulators (ICE).
- ▶ JTAG = Joint Test Access Group ([IEEE 1149.1](#))



JTAG Connector – Keil STR9 Board

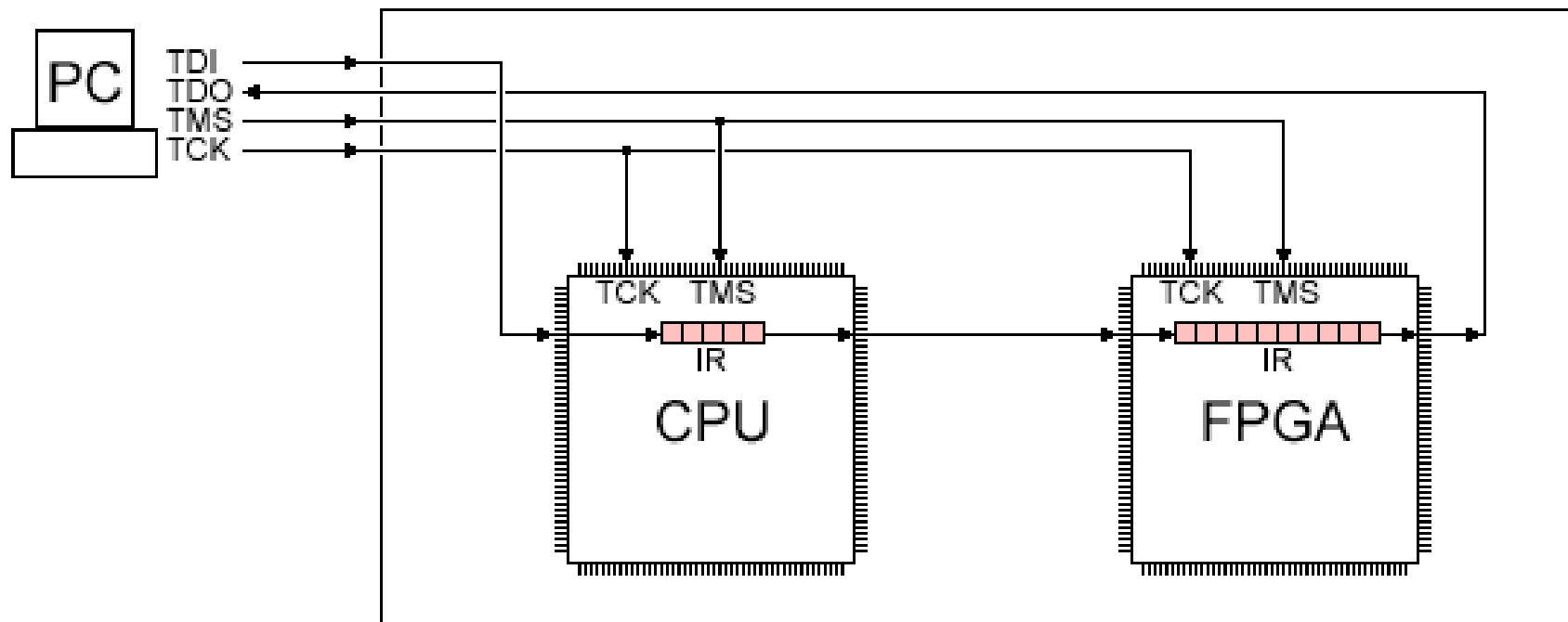


JTAG Scan Chain

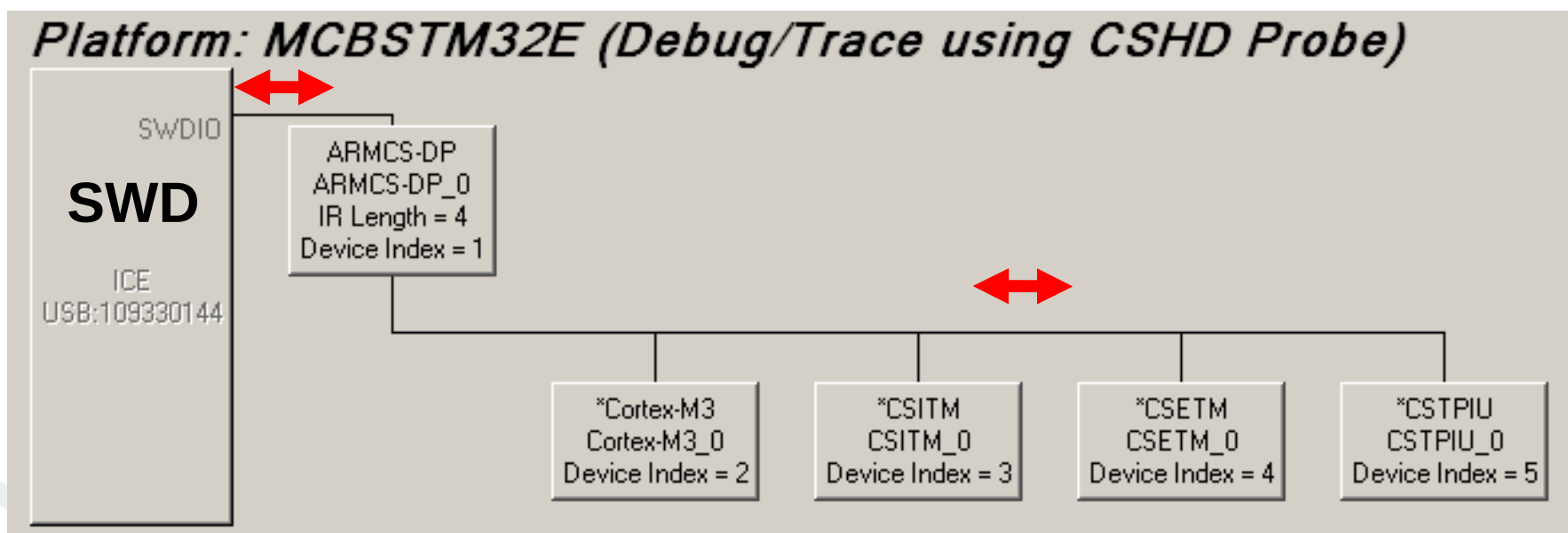
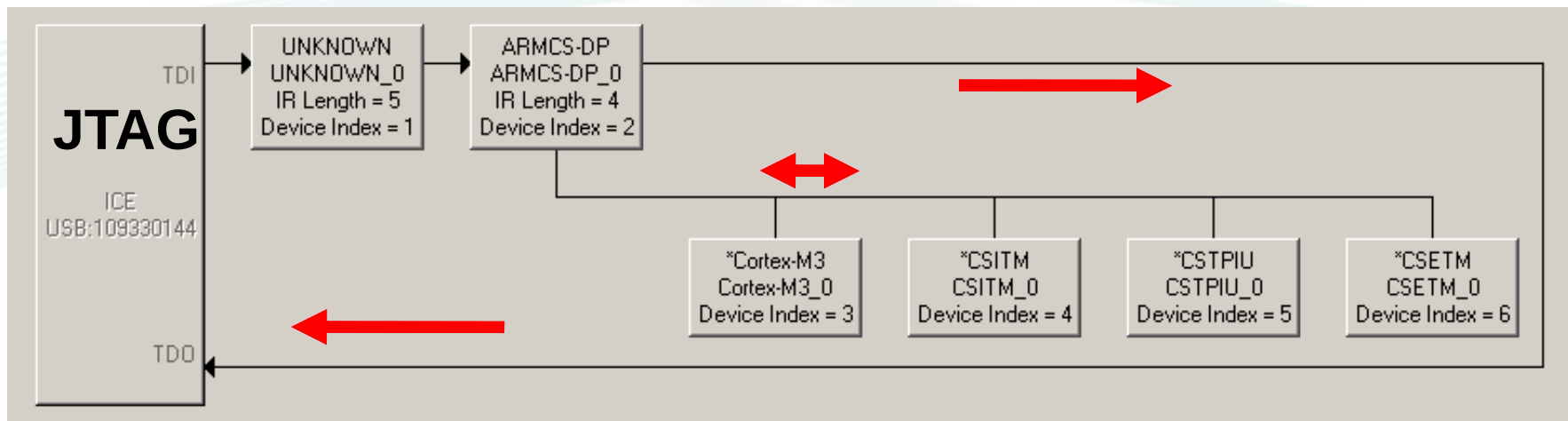
- ▶ Serial in and out in a “chain”
- ▶ Chip can have > 1 module or “TAP” “test access port”
- ▶ Such as CPU, Flash program, FPGA, ETM, etc.
- ▶ Can “Bypass” a TAP if not used. Saves time
- ▶ Data is clocked serially in and out by TCK
- ▶ TDI = + edge, TDO = - edge

Scan Chain – a couple of acronyms

- ▶ ID – ID Code. 32 bits. Provides info about device
- ▶ IR – Instruction Register – in bits
 - IR = All ones = bypass. (i.e. 11111 for 5 bit)
 - In and out of device is one flip-flop – BYPASS saves time

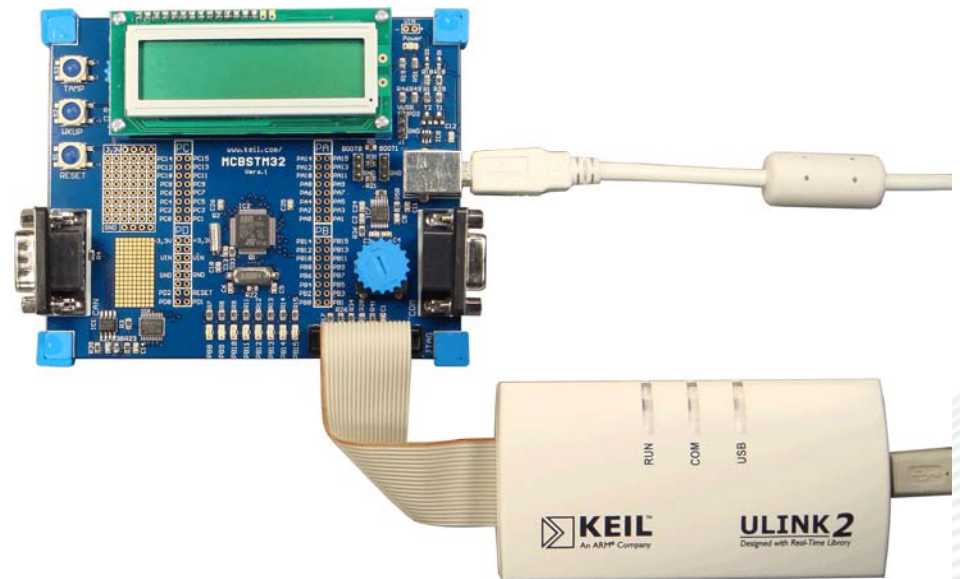


CoreSight on Cortex-M3 JTAG vs SWD:



SWD: Serial Wire Debug

- ▶ New to CoreSight. A 2 wire debug port.
- ▶ Very popular on Cortex processors.
- ▶ Bidirectional line (SWD) + clock (SWCLK). Simpler than JTAG.
- ▶ Same functionality as JTAG except no boundary scan.
- ▶ Most adapters now have this.
- ▶ Serial Wire Viewer goes hand-in-hand with SWD.
- ▶ SWD a.k.a. SW.



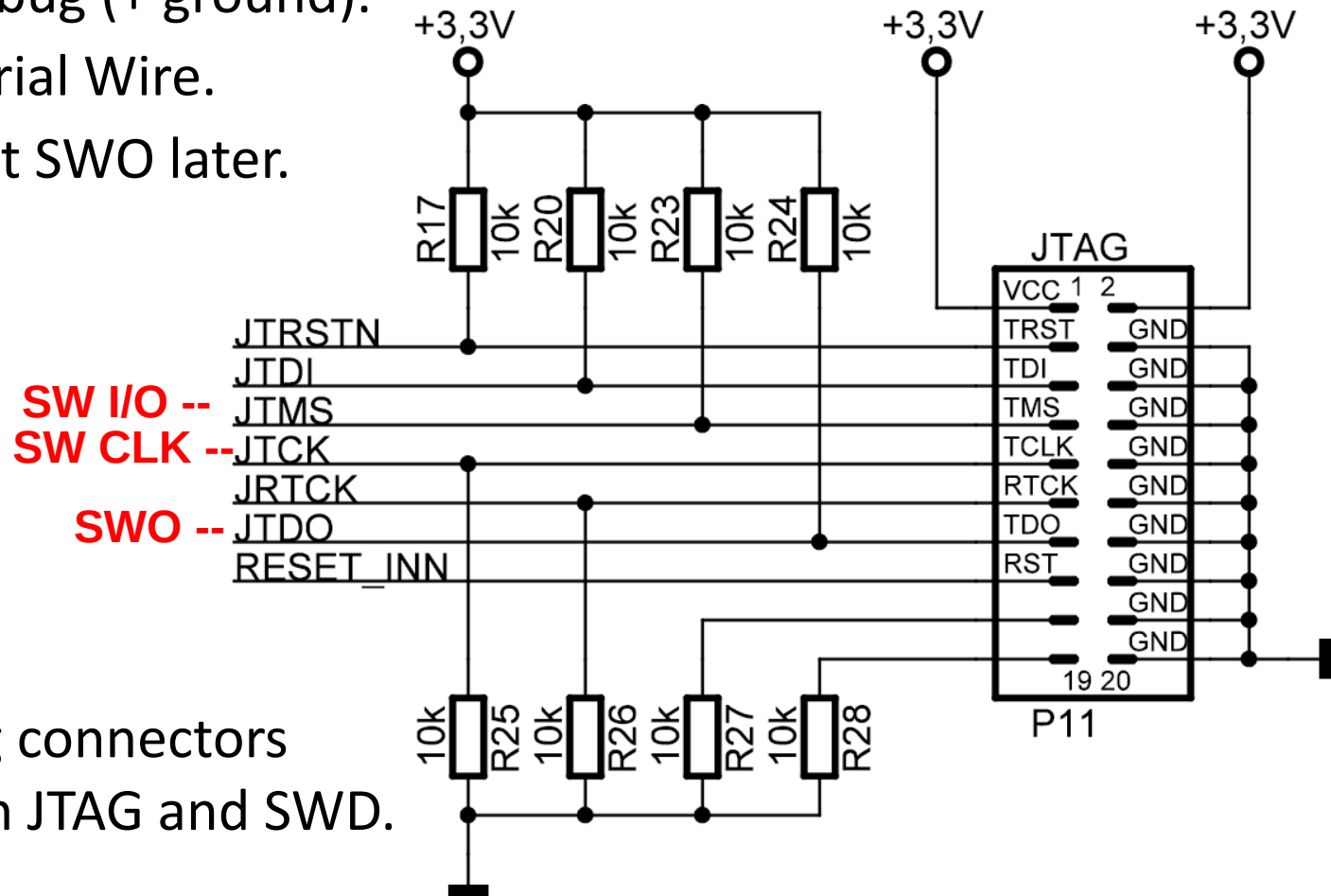
ARM

SWD pins ?

- ▶ Only 2 pins to control ARM core:
 - ▶ SWD – signal – shares TMS.
 - ▶ SWCLK – clock – shares TCK.
-
- ▶ SWO – Serial Wire Output – shares TDO.
 - ▶ SWD on your chip ? Check your datasheet.
 - ▶ Most common on Cortex-M0, M1, M3, M4, R4.

Serial Wire Connections:

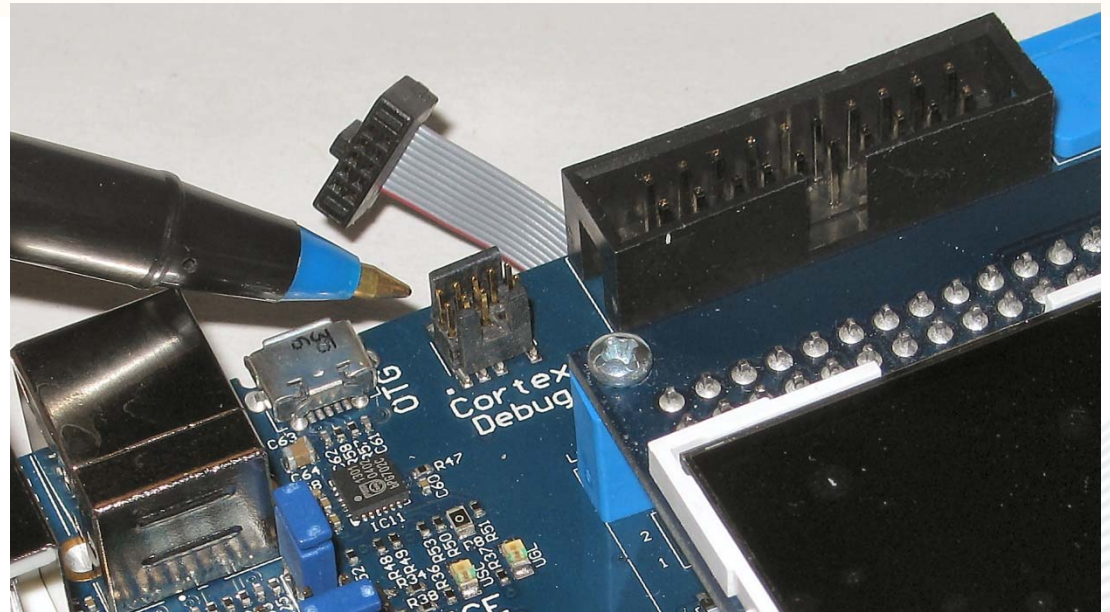
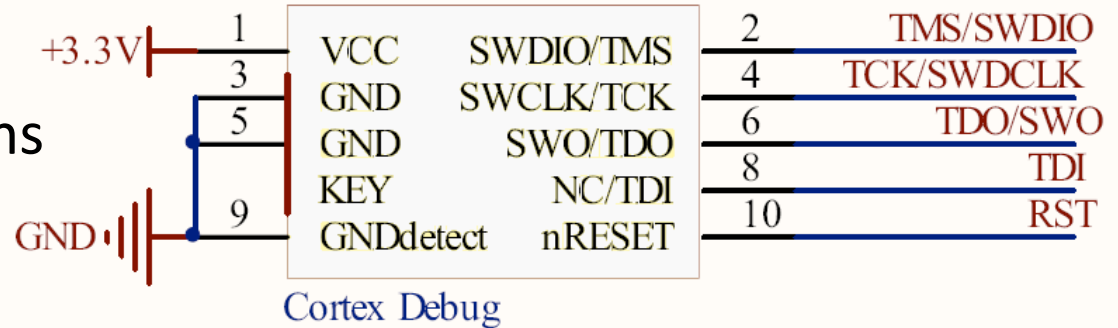
- ▶ Our JTAG diagram from before:
- ▶ 3 wire debug (+ ground).
- ▶ **RED** is Serial Wire.
- ▶ Talk about SWO later.



- ▶ All Debug connectors have both JTAG and SWD.

Cortex-M3 Hi-Density 10 pin

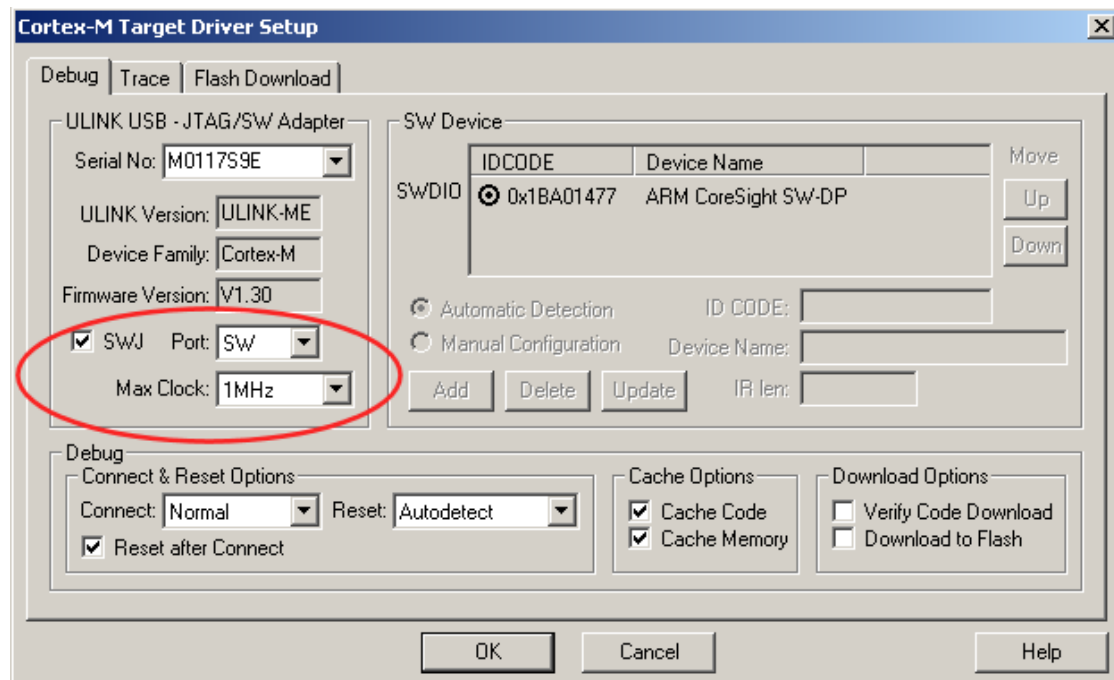
- ▶ 10 pin **NEW** connector.
- ▶ Note JTAG and SWD pins are present here too!
- ▶ No nTRST pin.
- ▶ SWD in CoreSight only.



TIP: DDI0314F_coresight_component_trm.pdf on www.arm.com

How do we use SWD ?

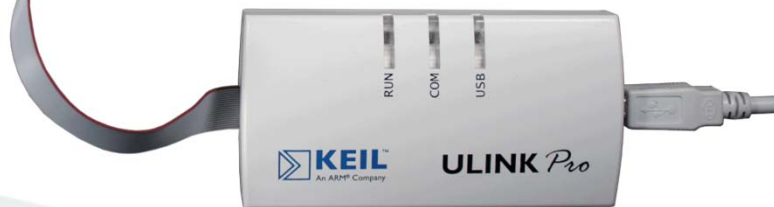
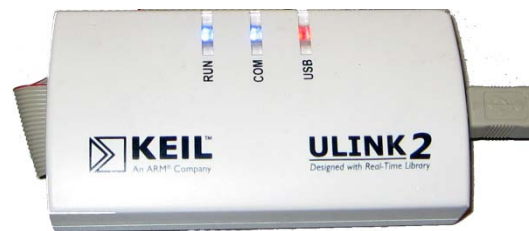
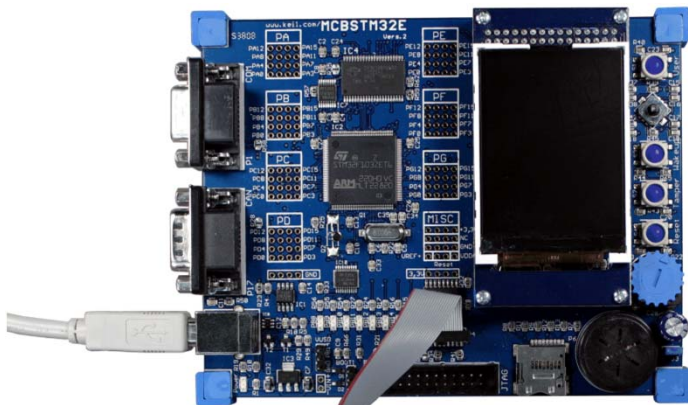
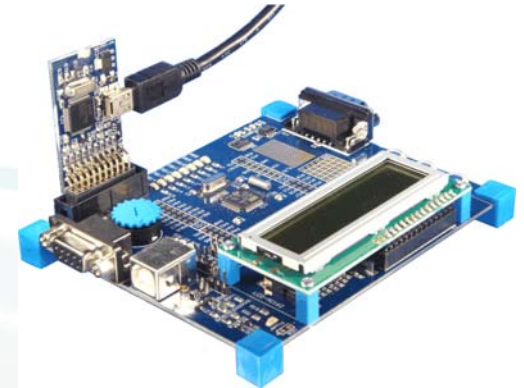
- ▶ End user – just plug in connector and select in the software.
- ▶ SWD in the core is automatically selected.
- ▶ Debugger will do necessary steps.
- ▶ This means debugger must be SWD capable. Most are.
- ▶ Select speed as in JTAG.



JTAG/SWD Adapters

► USB-JTAG adapter (for Flash programming too)

- ULINK2 - \$395 (*ULINK2 and ME - SWV only – no ETM*)
- ULINKpro - \$1,395 – Cortex-M3 SWV & ETM trace
- Signum JTAGJetTrace – Cortex-M3 SWV & ETM - \$1,500, ARM9+ - \$3,500 to \$4,500
- Segger J-Link and J-Trace are about the same prices.
- **Note:** If you use an ETM adapter – don't need a JTAG one too.



ARM

So what do we have so far ?

- ▶ JTAG debugging
 - ▶ Serial Wire Debug
 - ▶ These are used to connect your debugger with the core.
-
- ▶ What is next on our agenda ?
 - ▶ Tracing !
 - ▶ 1st ETM, then SWV.

Why Trace ?

- ▶ Shows history of instructions as they happened.
- ▶ This is called TRACE – collected in trace memory somewhere.
- ▶ Very high speed – as fast as the CPU...but there are limits.
- ▶ Can usually configure and view trace without stopping CPU.
- ▶ Often associated with Triggering and Filtering.
- ▶ These used to narrow your search. (else billions of records)
- ▶ Some have internal memory, use ETB or are Streaming Trace.

ETM: Embedded Trace Macrocell:

- ▶ For high-end users or those with those really nasty problems.
- ▶ Cost is \$1,500 to \$4,000 +. For some customers, it is worth it.
- ▶ ETM has an easy, quick payback time.
- ▶ Displays where the program went and what data was read or written plus when exceptions happened.
- ▶ Non-intrusive. Real-time. Uses no CPU cycles. Most customers will know if they need ETM.
- ▶ Controlled through JTAG or SWD.
- ▶ Data sent out the special port: Trace Port.

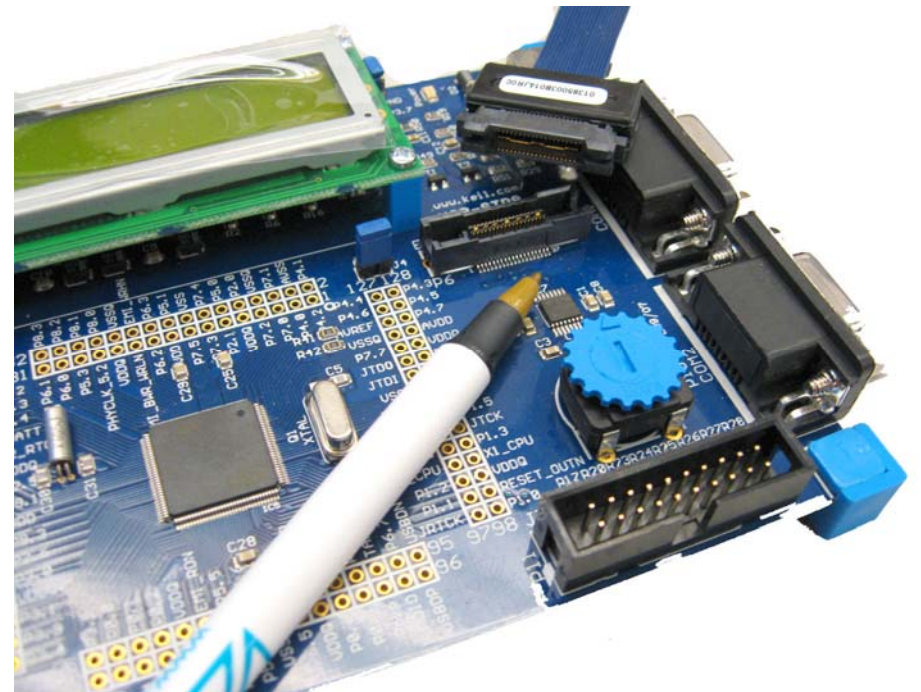
What comes out the ETM ?

- ▶ Instruction Trace:
 - Address of executed instruction indicating program flow.
 - Instruction will be disassembled by debugger.
- ▶ Data Trace: (only EmbeddedICE – CoreSight uses SWV)
 - The data value and its address resulting from a load or store.
 - Instruction causing this will be displayed on another packet.
 - Can send out only address or data to save time.
 - Why ?
- ▶ Can see/record either everything out the trace or filter out what frames you don't want.
- ▶ A timestamp is provided.

How do we connect to ETM ?

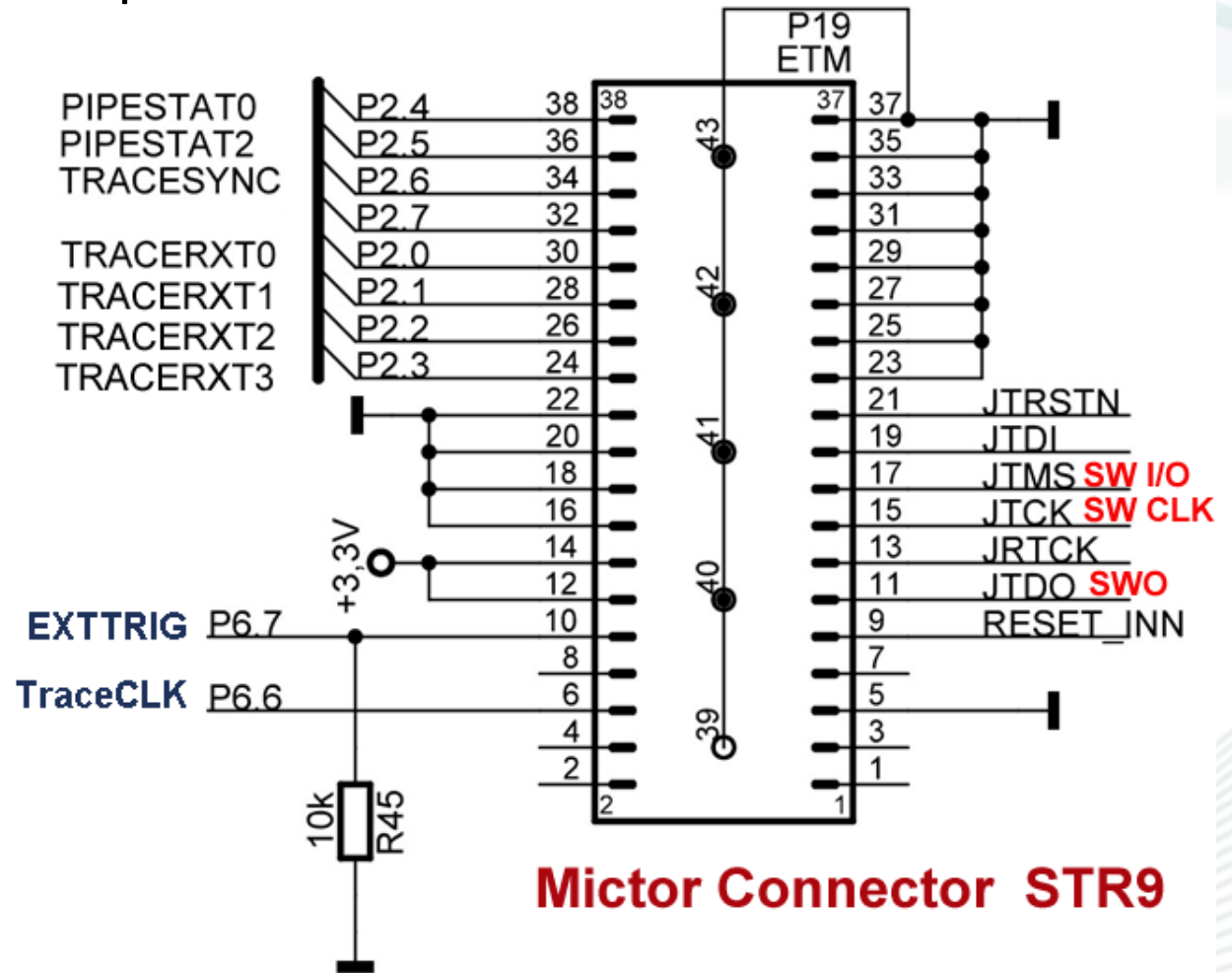
- ▶ ETM is 1, 2, 4, 8, 16, 24 or 32 bit output.
- ▶ Uses a special Mictor connector.
- ▶ 1 Mictor up to 16 bit.

- ▶ JTAG/SWD signals on Mictor too
- ▶ Speed to 480 mHZ or more <?>



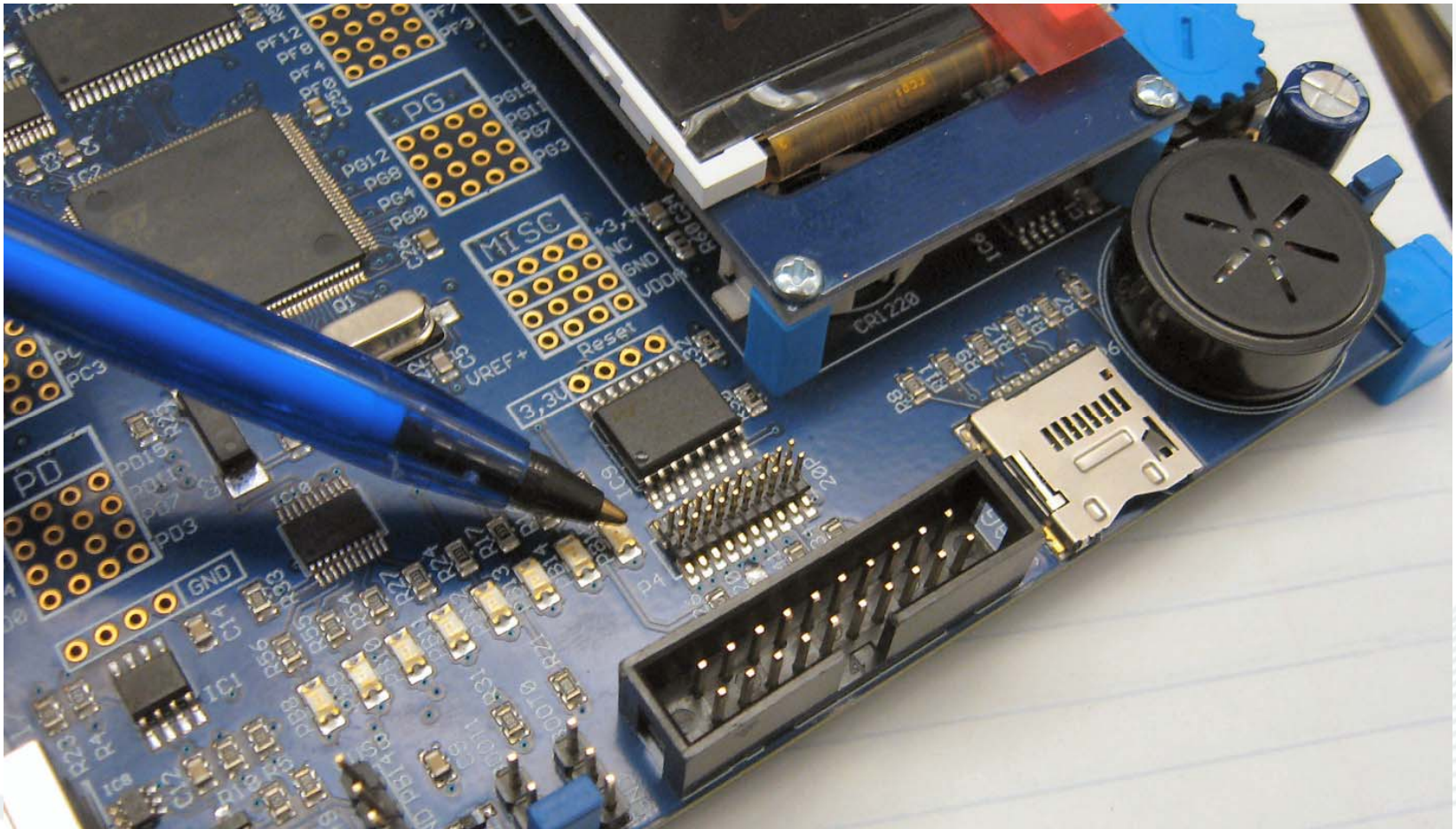
ETM Pins...Mictor connector

- ▶ 38 pin Mictor for fast chips – STR9.
- ▶ Note JTAG and SW pins. are present.
- ▶ Trace clock.
- ▶ Can use for M3/M4



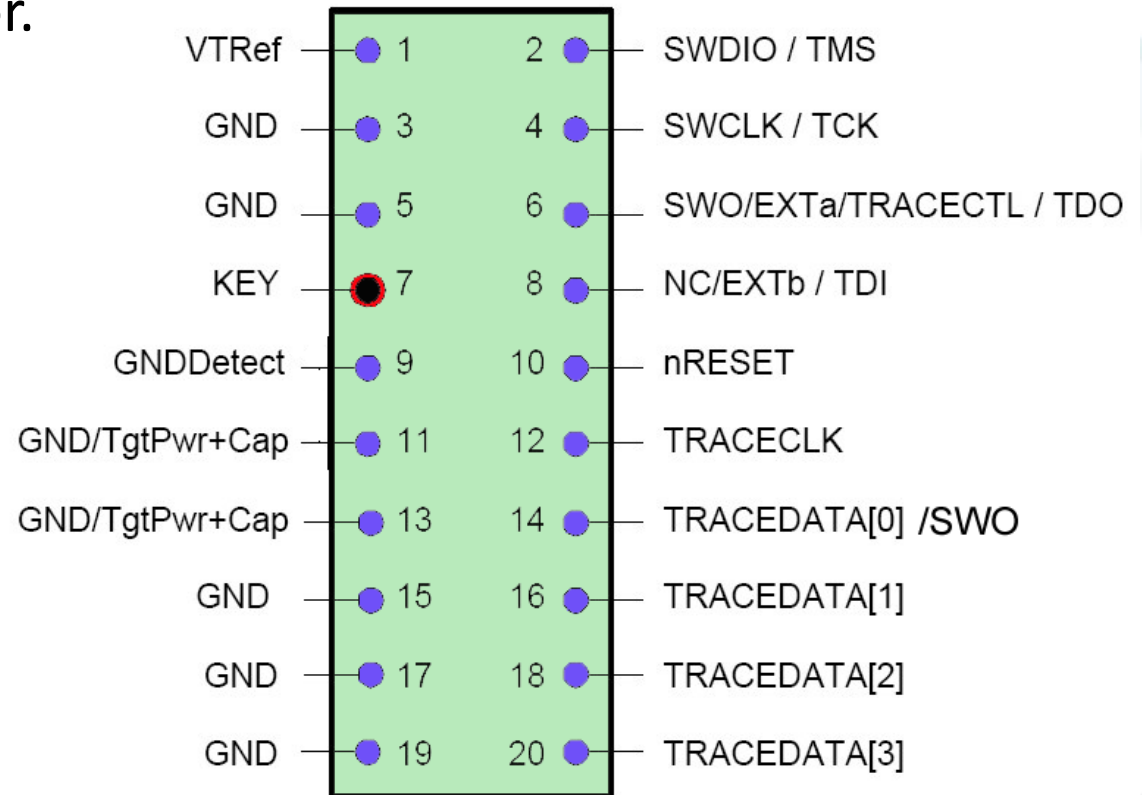
More on the ETM...

- ▶ Cortex-M3, M4 can use newer, smaller connector.
- ▶ This is on Keil MCBSTM32E board.



ETM Cortex-M3/M4 Hi-Density 20 pin

- ▶ 20 pin **NEW** connector.
- ▶ Has ETM pins.
- ▶ JTAG, SW
- ▶ Note JTAG pins are present here too!
- ▶ Trace simplified.
- ▶ TraceCLK PCB layout more critical than trace data.



The pinout of the ARM high-density 20-pin Cortex connector. Top view.

TIP: [DDI0314F_coresight_component_trm.pdf](#) on www.arm.com

www.samtec.com/ftppub/pdf/ftsh_mt.pdf



Where is ETM on the chip pins ?

- ▶ ETM shares pins – often not the best ones....
 - ▶ But usually GPIO...but not always See datasheet.
 - ▶ If need ETM – please leave these GPIO open...If you can....
 - ▶ If customer might use/need ETM – avoid these pins.
 - ▶ Can be a big problem.... A show stopper for tracing...
-
- ▶ NAND Flash memory controller.
 - ▶ Ethernet controller pins.
 - ▶ Check potential devices to see what is shared.
-
- ▶ ETM is very fast – as fast as the core.
 - ▶ If the core goes into the weeds – ETM will still function.

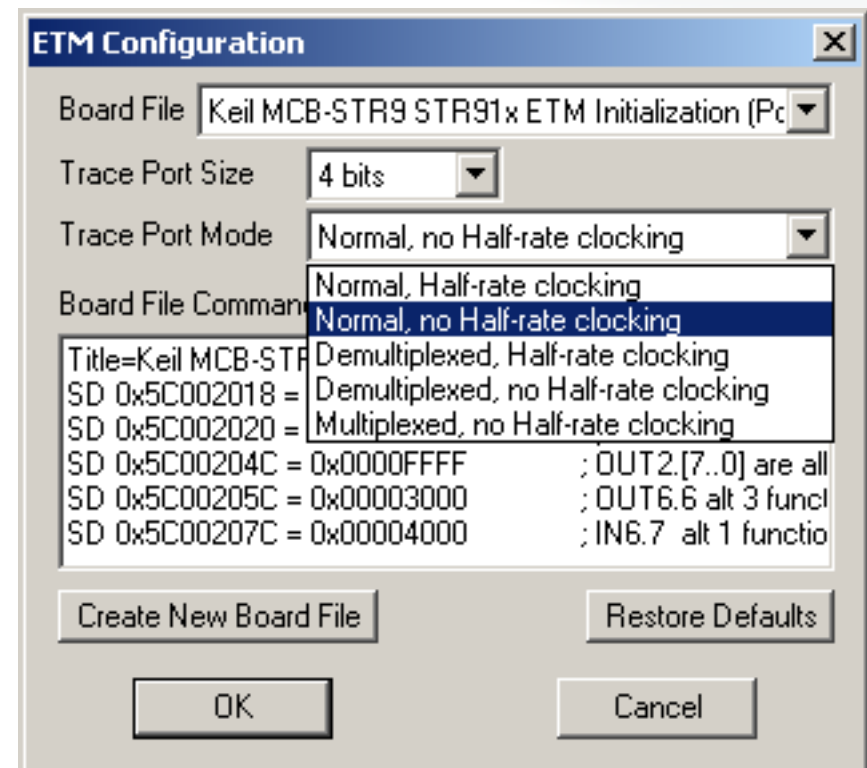
ETM: Embedded Trace Macrocell:

- ▶ ETM provides instructions: program flow is obvious.
- ▶ PC, data R/W's, Exceptions, timestamp.
- ▶ Triggers and Filter Can filter out stuff we don't want to see.
- ▶ Assembly and C source code is displayed.

Instruction Trace											
Filter: All											
#	Type	Flag	Num	PC	Opcod	Instruction	Source Code	Address	Data	Cycles	Time[s]
1048553	ETM			0x080004B4	6010	STR r0,[r2,#0x00]					
1048554	ETM			0x080004B6	4B1D	LDR r0,[pc,#116] ; @0x080005...					
1048555	Exception Exit		15							288005127	4.00007121
1048556	Exception Return		0							288005135	4.00007132
1048557	ETM			0x080004B8	8800	LDRH r0,[r0,#0x00]					
1048558	ETM			0x080004BA	EBB12F20	CMP r1,r0,ASR #8					
1048559	ETM			0x080004BE	D312	BCC 0x080004E6					
1048560	ETM			0x080004E6	4B12	LDR r0,[pc,#72] ; @0x08000530	47: ADC1->CR2 = (1 << 22); /* start SW conversion */				
1048561	ETM			0x080004E8	6800	LDR r0,[r0,#0x00]					
1048562	ETM			0x080004EA	F4400080	ORR r0,r0,#0x400000					
1048563	ETM			0x080004EE	4911	LDR r1,[pc,#68] ; @0x08000534					
1048564	ETM			0x080004F0	F8C10408	STR r0,[r1,#0x408]					
1048565	ETM			0x080004F4	BD10	POP {r4,pc}	48: }				
1048566	ETM			0x080003DE	8800	LDRH r0,[r0,#0x00]					
1048567	ETM			0x080003E0	4284	CMP r4,r0					
1048568	ETM			0x080003E2	D004	BEQ 0x080003EE					
1048569	ETM			0x080003EE	4625	MOV r5,r4	133: AD_print = AD_value; /* Get unscaled value for printout ...				
1048570	ETM			0x080003F0	4B13	LDR r0,[pc,#76] ; @0x08000440	135: if (clock_1s) {				
1048571	ETM			0x080003F2	7800	LDRB r0,[r0,#0x00]					
1048572	ETM			0x080003F4	B130	*CBZ r0,0x08000404					
1048573	ETM			0x080003F6	2000	MOVS r0,#0x00	136: clock_1s = 0;				
1048574	ETM			0x080003F8	4911	LDR r1,[pc,#68] ; @0x08000440					
1048575	ETM			0x080003FA	7008	STRB r0,[r1,#0x00]					
1048576	ETM			0x080003FC	4629	MOV r1,r5	137: printf ("AD value = 0x%03x\n\r",AD_print);				

ETM Operating Modes:

- ▶ Need to set this in debugger (some automatic).
- ▶ Set processor pins to ETM mode
- ▶ Configure ETM
- ▶ Can then capture ETM trace records.
- ▶ Display in screen.
- ▶ Should not have to stop program execution to view trace.

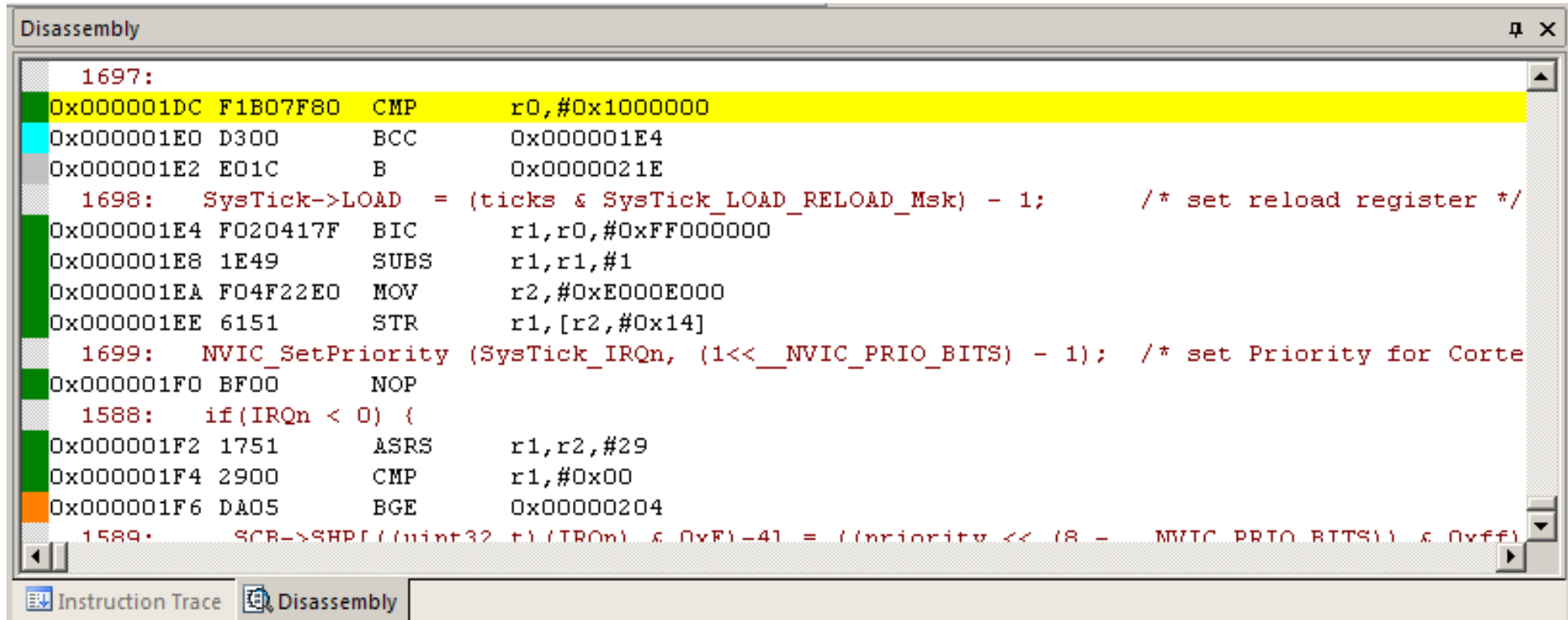


So...is confusing – how do I set all these ?

- ▶ Check your data sheet.
- ▶ Trial and error with best guess....
- ▶ Best: call your emulator manufacturer tech support.
- ▶ If not set right – probably will not get any traces. Or junk.
- ▶ Look for indication of trace clock value. This is important.
- ▶ Debugger sets this value. ETM can return this value.

Code Coverage: provided by ETM

- ▶ Was every instruction executed and tested ?
- ▶ You do not want untested code in your product that might get activated under some strange condition !
- ▶ Can save to a file.

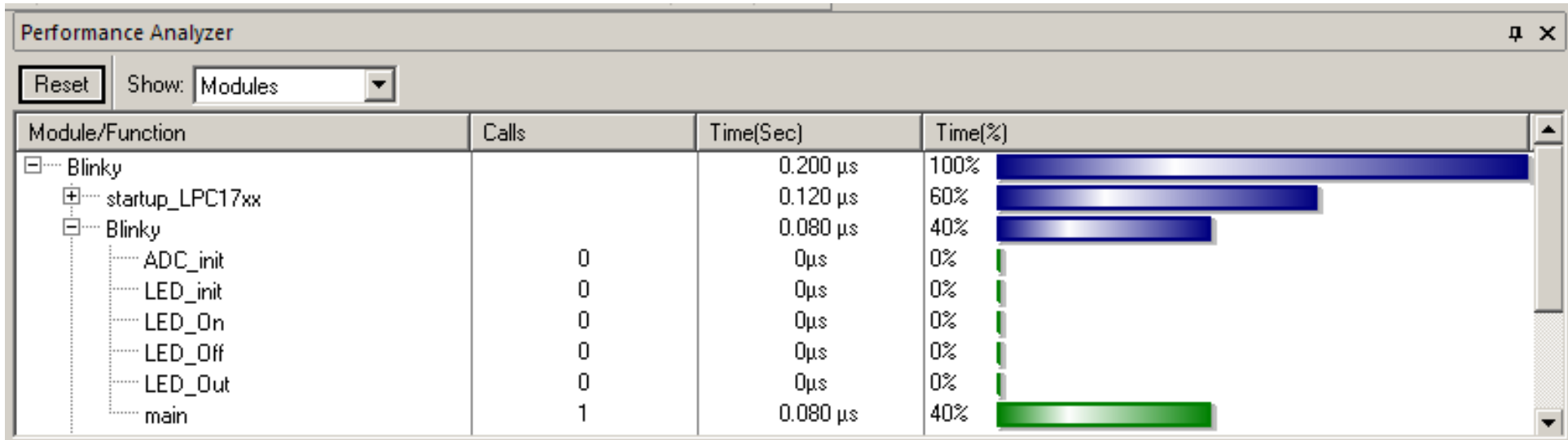


The screenshot shows a disassembly window titled "Disassembly" with a list of ARM instructions. The instructions are color-coded: yellow for the first instruction, cyan for the second, and green for the others. The instructions are as follows:

```
1697: 0x000001DC F1B07F80 CMP r0,#0x1000000
0x000001E0 D300 BCC 0x000001E4
0x000001E2 E01C B 0x0000021E
1698: SysTick->LOAD = (ticks & SysTick_LOAD_RELOAD_Msk) - 1; /* set reload register */
0x000001E4 F020417F BIC r1,r0,#0xFF000000
0x000001E8 1E49 SUBS r1,r1,#1
0x000001EA F04F22E0 MOV r2,#0xE000E000
0x000001EE 6151 STR r1,[r2,#0x14]
1699: NVIC_SetPriority (SysTick_IRQn, (1<<__NVIC_PRIO_BITS) - 1); /* set Priority for Corte
0x000001F0 BF00 NOP
1588: if(IRQn < 0) {
0x000001F2 1751 ASRS r1,r2,#29
0x000001F4 2900 CMP r1,#0x00
0x000001F6 DA05 BGE 0x00000204
1589: SCR->SHPR[(uint32_t)(IRQn) & 0xF]-41 = ((priority << (8 - NVIC_PRIO_BITS)) & 0xff)
```

Performance Analyzer: provided by ETM

- ▶ Where did your program spend all its time ?
- ▶ Can often be quite illuminating and unexpected.
- ▶ ETM provides the information.
- ▶ Could use PC Samples but must run a longer time.

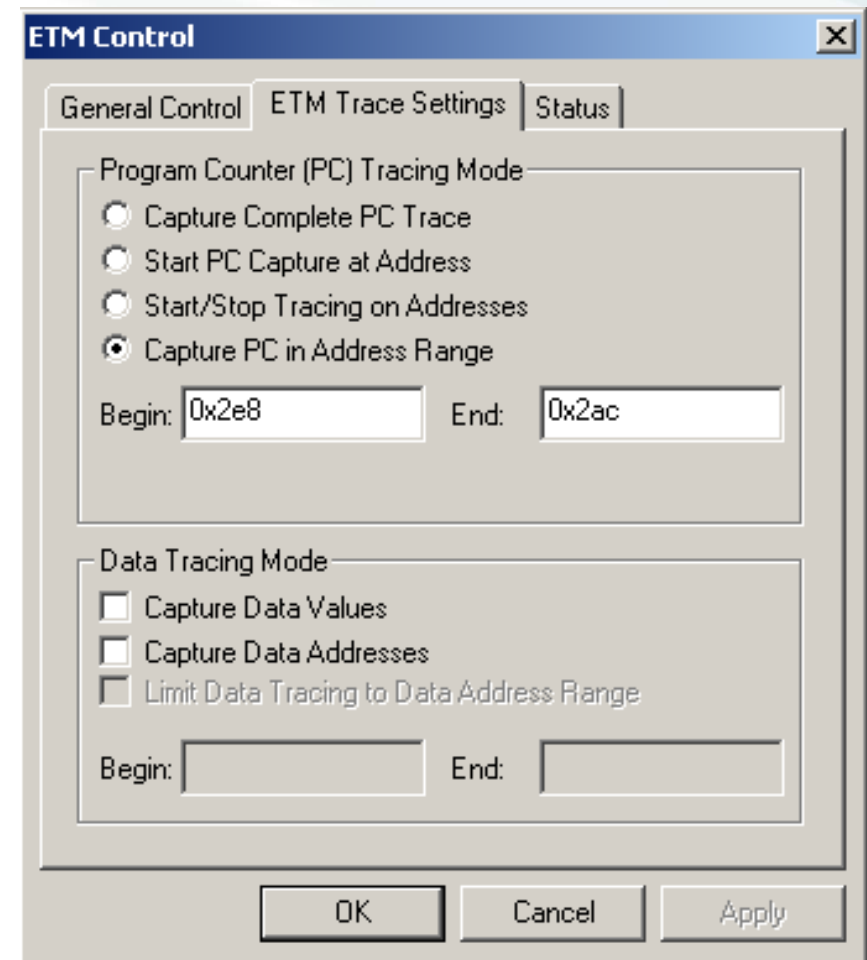


ETM Triggers and Filters

- ▶ **Triggers** – stop either/both program exec or trace collection when some specified event happens
- ▶ **Filters** – collect or see only the specified data you want to see
- ▶ **Pre- or post- filtering:**
(don't want to fill up expensive buffer with junk)
- ▶ Triggers and filters allow you to zero in on your problem
- ▶ Effective and easy to set T & F
- ▶ Takes a bit of practice to be really efficient
- ▶ Can do a lot with a little training

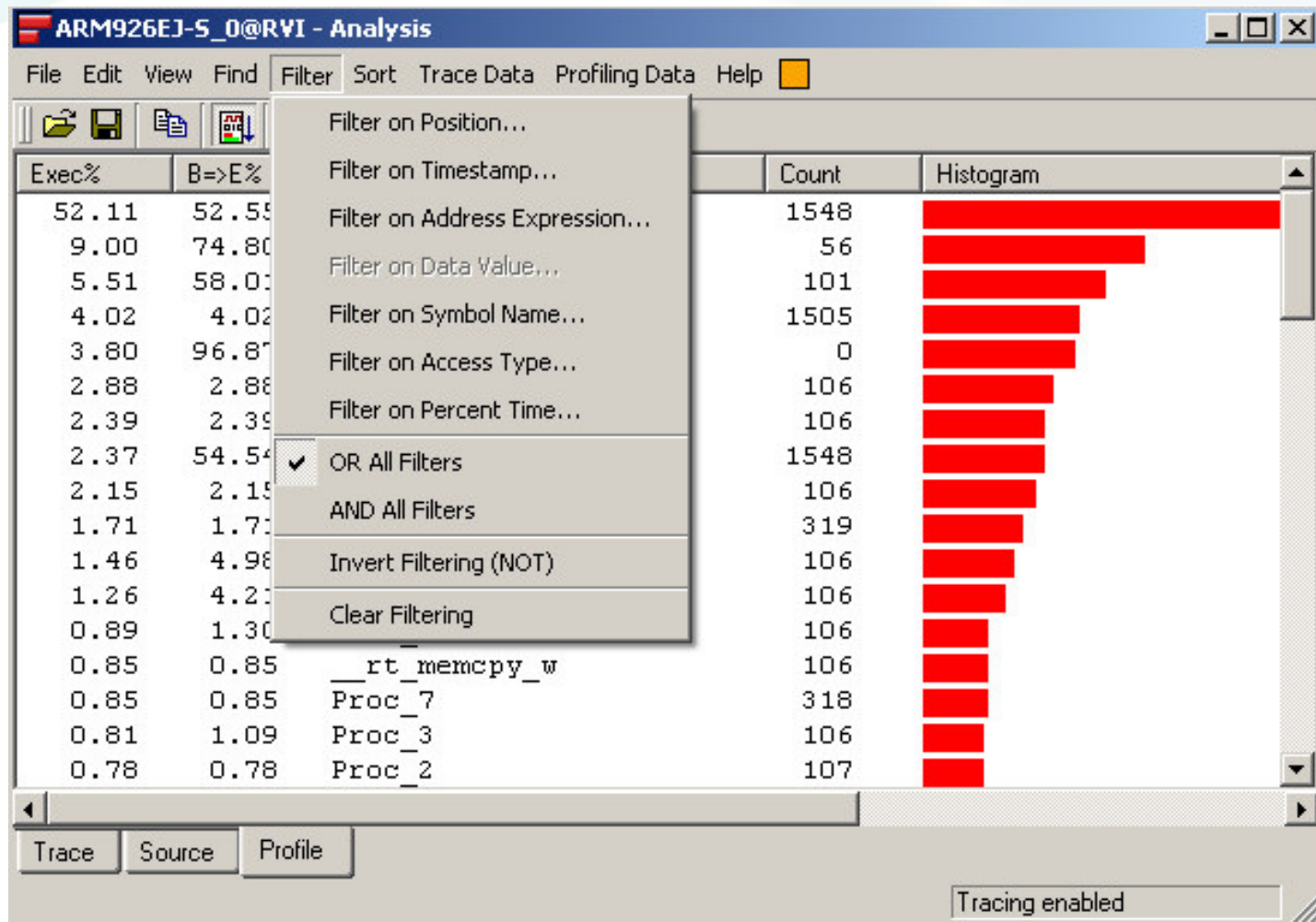
ETM Triggers and Filters

- ▶ Here is Signum's tracing configuration window. Easy to use.
- ▶ Buffers in debuggers usually 1 – 4 Mbytes.
- ▶ Others can stream trace frames into your PC memory for a long, long time.



ARM RVDS Trace Filters

- ▶ ARM RVD Profiler + Filters shown.



ETB: what is this ?

- ▶ When frames sent out ETM port. Very fast - \$\$ debugger.
- ▶ What if we could access ETM through JTAG or SWD ?
- ▶ Could use ordinary (and cheaper) JTAG/SWD adapters.
- ▶ This is ETB – Embedded Trace Buffer.
- ▶ Is memory – an ETM buffer.
- ▶ Size set by chip designer.
- ▶ 2, 4 or 8 Kbytes are common.
- ▶ Memory accessible by user program via data bus.
- ▶ Can access ETM for test or diagnostic purposes.

ETB Advantages

- ▶ No expensive ETM hardware required.
- ▶ Any JTAG adapter will work. Debugger has to have ETB support.
- ▶ If core is too fast – is the only way to get trace data.
- ▶ Can be triggered and filtered.
- ▶ You can easily access ETB memory with your application programs !!!!!!! PHONE HOME !!!
- ▶ **Note:**
 - ▶ EmbeddedICE: ETB accessed like any other memory.
 - ▶ CoreSight: ETB memory accessed through a register.

ETB Disadvantages

- ▶ Memory size is very small. (2 to 8K)
- ▶ Can't save large sections of the core run.
- ▶ Must use filters and triggers to focus on data.
- ▶ Might have to stop the program to read data. (depends)
- ▶ ETB memory might be irresistible to “steal”.
- ▶ Then can't use ETB to debug.

ETM & ETB Summary

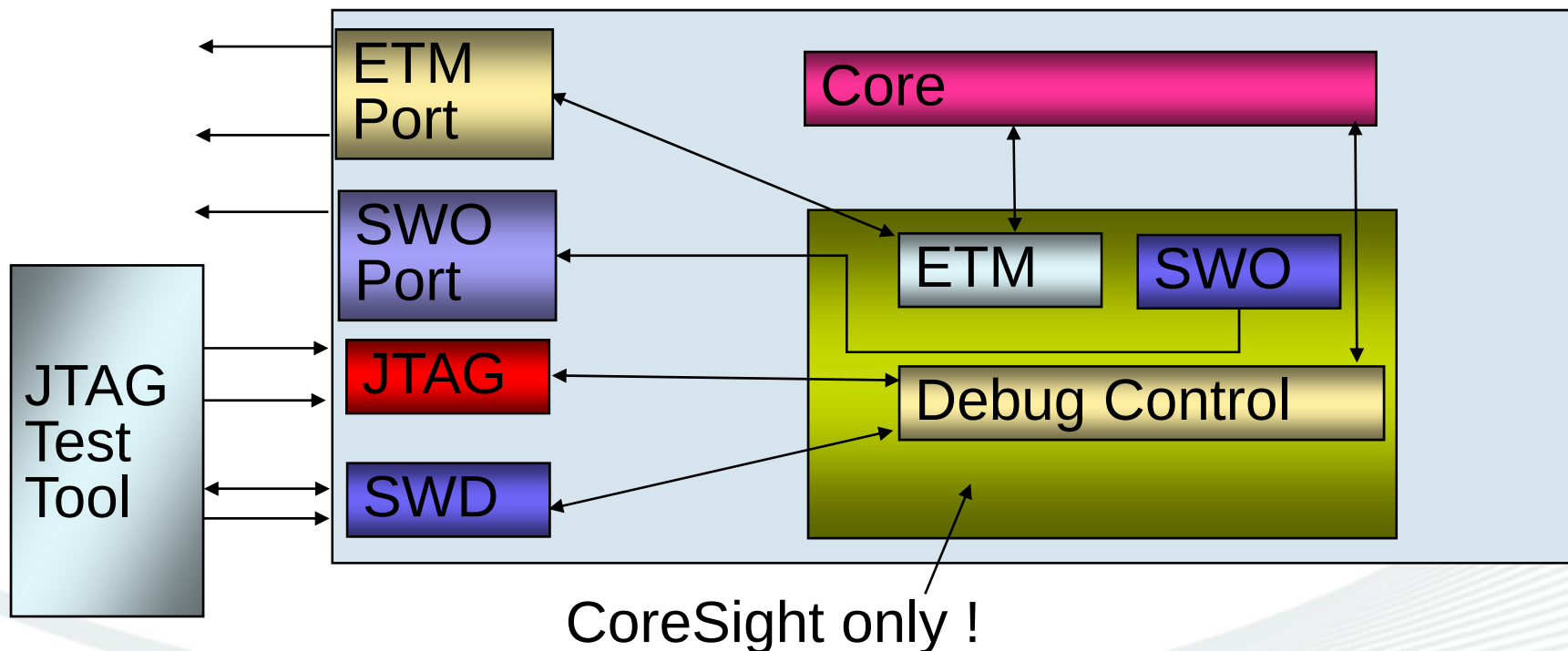
- ▶ ETM is very, very fast. This is why ETM adapters expensive.
- ▶ Displays all PC values, data reads & writes...all timestamped.
- ▶ Can filter and trigger.
- ▶ ETB a low cost way to get ETM info out JTAG port.
- ▶ Cortex-Mx and Cortex-A9 don't have data tracing out ETM.
- ▶ Use Serial Wire Viewer for data R/Ws and exceptions.
- ▶ But – check your datasheet to see what is implemented.

Serial Wire Viewer - Introduction

- ▶ Only on Cortex microcontrollers:
 - Serial Wire Debug (SW-DB) to connect to core. (2 wires)
 - Serial Wire Viewer (SWV) module in microcontroller.
 - Serial Wire Output (SWO) to output the data. (1 wire)
 - SW-DB and SWO are on the standard JTAG connector.
- ▶ STMicroelectronics, Luminary, NXP, Toshiba ++ many more Cortex cores have SWV. Standard for Cortex-M3, M4.

SWO Block Diagram

- ▶ Can still have JTAG but – must use SWD for SWO.
- ▶ ETM + SWO becoming popular: ST, NXP, Toshiba, Fujitsu...
- ▶ SWO is only 1 serial pin output. Limiting.



Serial Wire Viewer: SWV

- ▶ Plugs into the regular JTAG connector.
- ▶ Non-intrusive. (except for ITM)
- ▶ Can view variables and peripherals in Real Time !
- ▶ Plus exceptions ! All in real-time !
- ▶ All other ARM debug methods are still available.
- ▶ Can have statistical Profile analysis.
- ▶ Code Coverage a problem – would have to run a long time.
- ▶ Instruction sampling and data tracing.
- ▶ Real-time viewing of RTOS data !
- ▶ SWV can come out the SWO pin or Trace Port.

Serial Wire Viewer has these features:

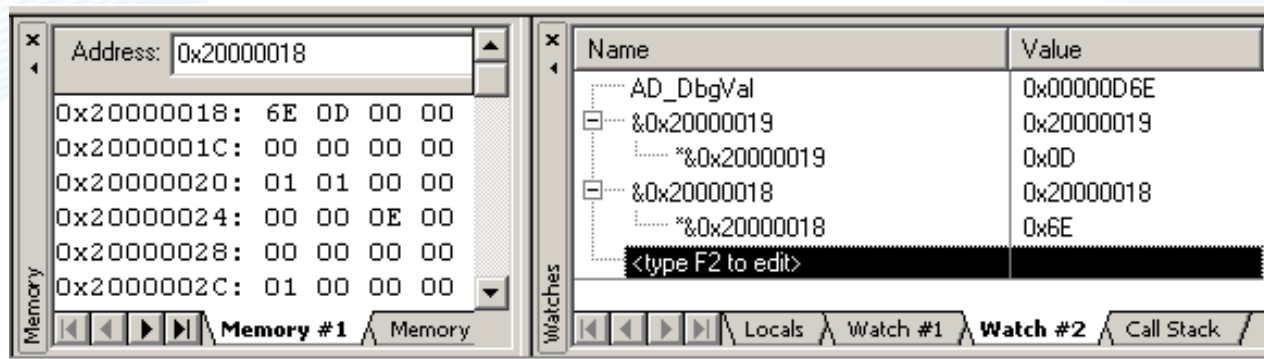
- ▶ Data Read and Data Write tracing...*in real time* !
- ▶ Can view data in graph, watch, memory or Serial Wire Viewer, Trace Records windows.
- ▶ Hardware Breakpoints and Watchpoints.
- ▶ ETM trigger (if the processor is so equipped).
- ▶ Program Counter (PC) or Data address sampler registers (Four 32 bit registers).
- ▶ ITM Viewer (Instruction Trace Macrocell): **printf** debugging

More on SWV:

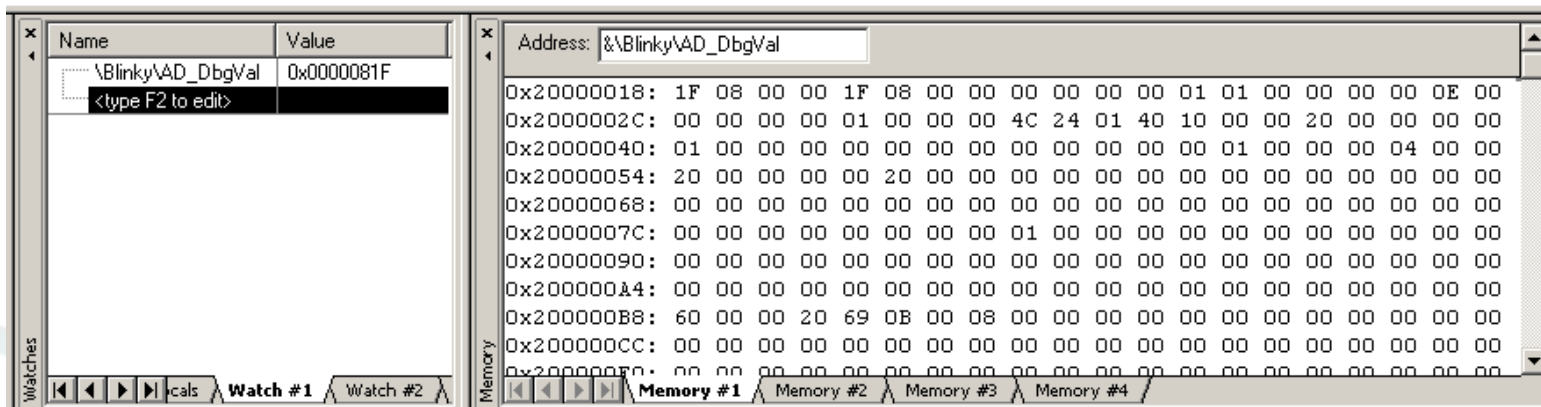
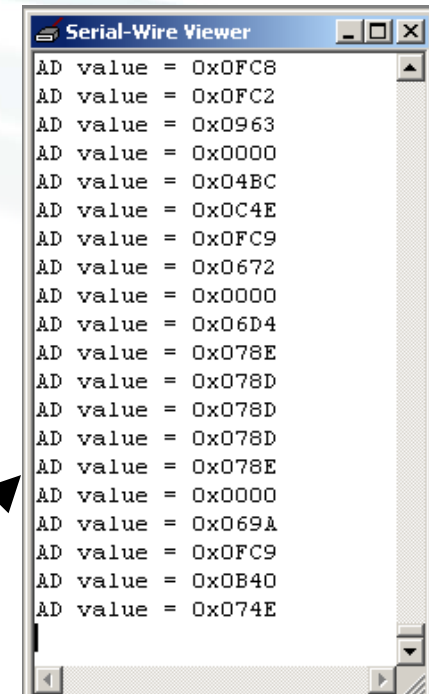
- ▶ In addition, the SWV trace module has counters for:
 - # of CPU clock cycles (32 bits).
 - Total cycles spent in interrupt (exception) processing (8 bit).
 - # of cycles processor is sleeping (8 bit).
 - Total cycles spent in load/store operations.
 - # of instruction cycles (8 bit).
 - # of folded instructions count (8 bits).
- ▶ Is useful to see how many extra cycles are used for instructions.

Serial Wire Viewer SWV

- ▶ View variables many different ways:

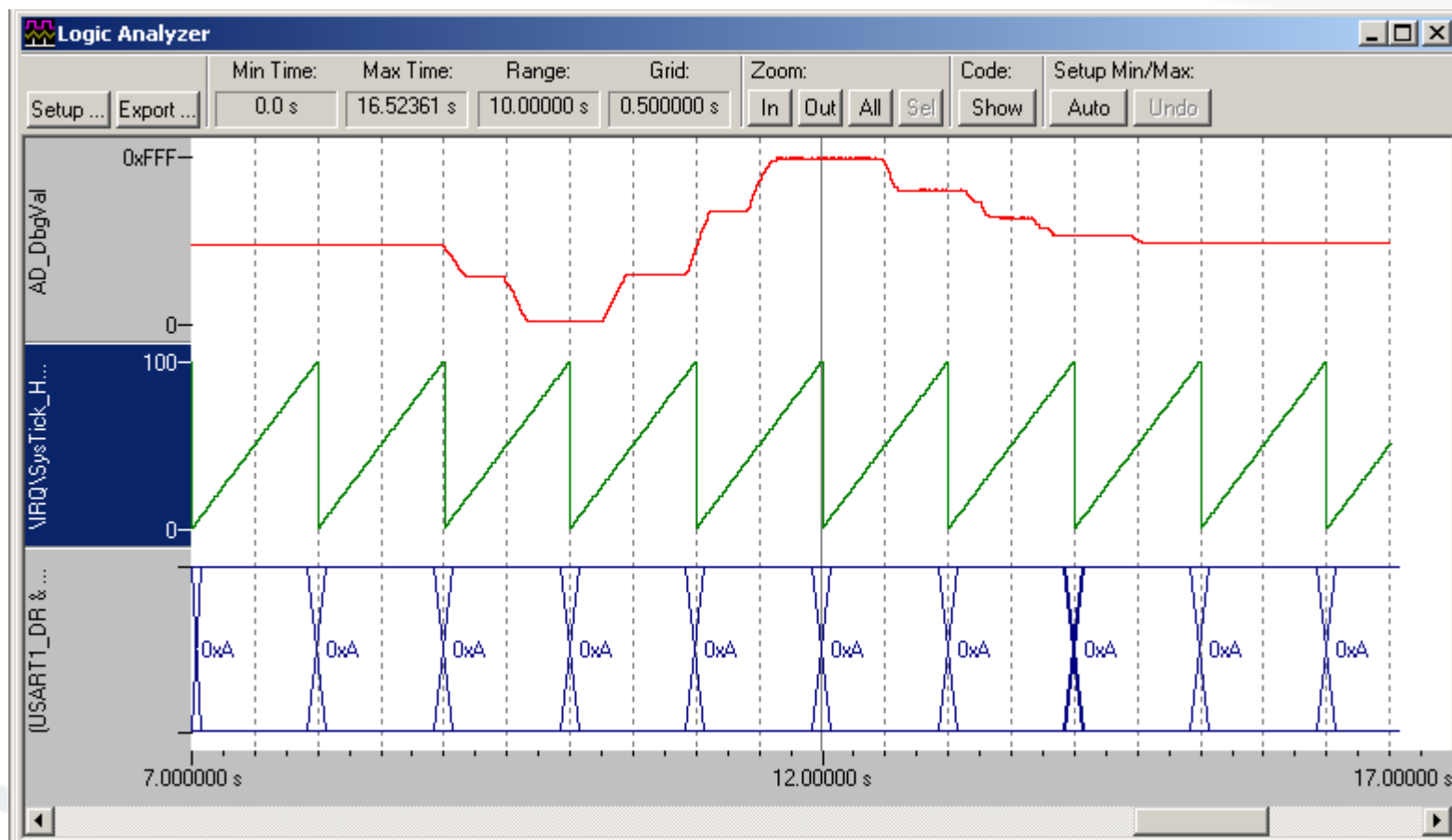


Open this in View/Serial Window/ITM Viewer.
Customer software writes to ITM: comes out SWO.



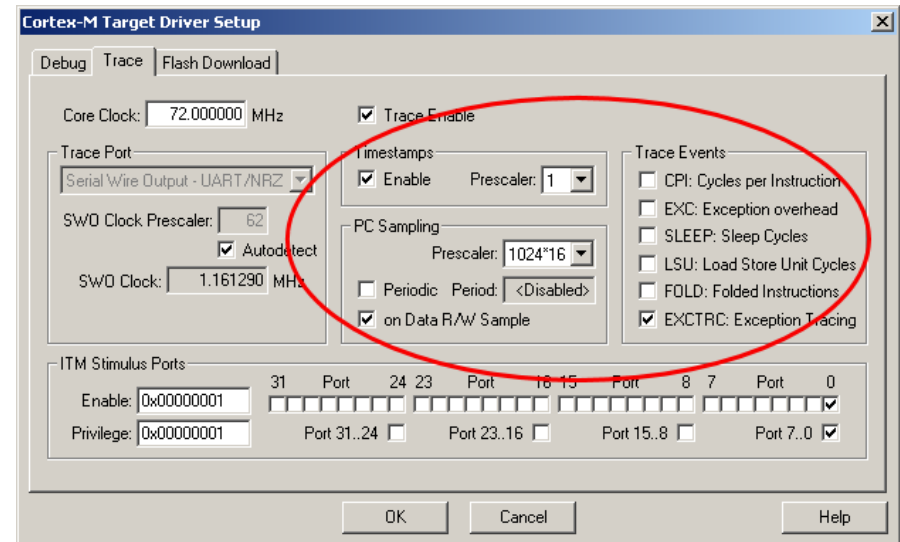
Serial Wire Viewer SWV

- ▶ Logic Analyzer Window:
- ▶ Pot on MCBSTM32™ is being rotated.



Trace Windows

- ▶ Three Trace Windows:
 - 1) Trace Records
 - 2) Exceptions
 - 3) Counters
- ▶ Displays types of information from the trace.
- ▶ Cortex-M Target Driver Setup window.
- ▶ Will be captured.
- ▶ If we select too many variables, this might cause a data overflow.



Trace Records

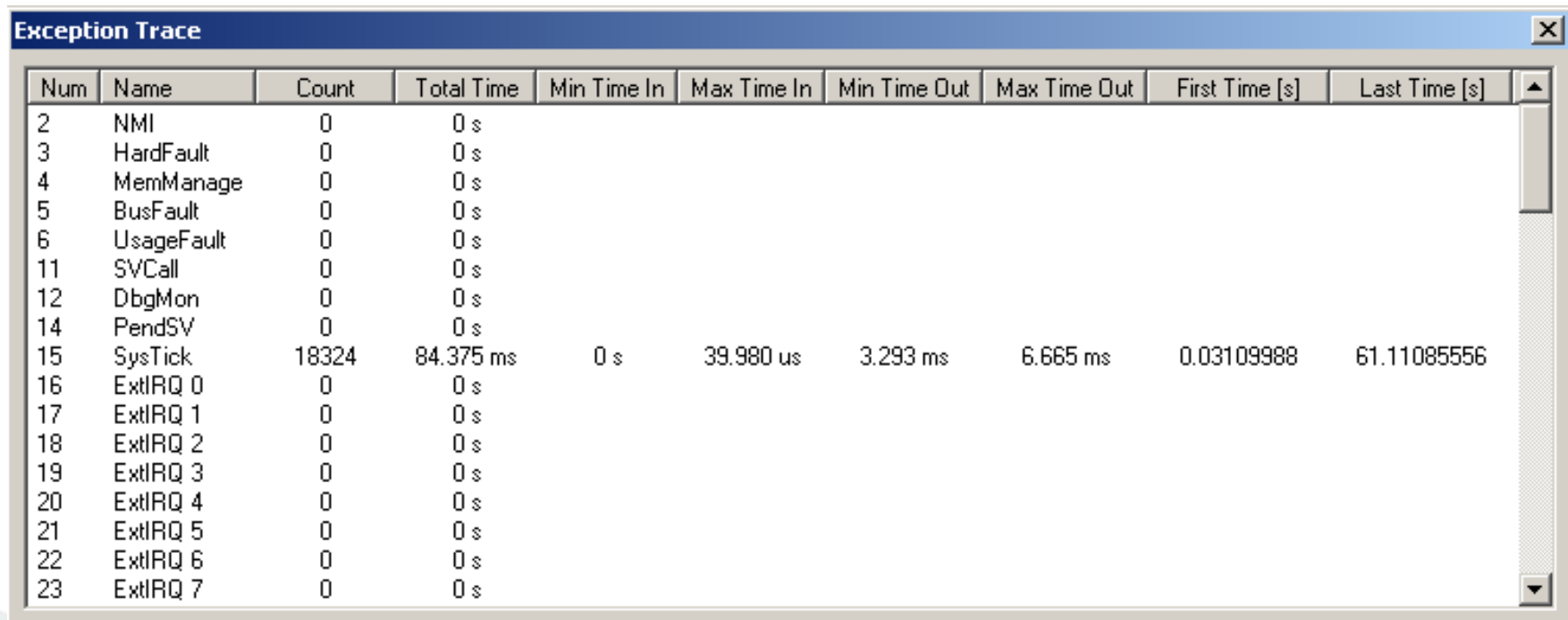
- ▶ Timestamp, PC sample, Read/Write PC, Exceptions, ITM.
- ▶ Filter window to remove certain lines. Post.
- ▶ Can change filters if CPU is running.
- ▶ Time delay and lost cycles noted.

- ✓ Counter Events
- ✓ Exceptions
- ✓ PC Samples
- ITM Events
- ✓ Data Reads
- Data Writes

Trace Records								
Type	Ovf	Num	Address	Data	PC	Dly	Cycles	Time[s]
Exception Return	X	0				X	6256132607	86.89073065
Exception Entry		15					6256844585	86.90061924
Data Write	X		20000004H	00000008H	080001A2H	X	6256852613	86.90073074
Exception Return	X	0				X	6256852613	86.90073074
Exception Entry		15					6257564593	86.91061935
Data Write	X		20000004H	00000009H	080001A2H	X	6257572619	86.91073082
Exception Return	X	0				X	6257572619	86.91073082
Exception Entry		15					6258948782	86.92984419
Data Read			20000004H	00000009H	0800019EH	X	6258951262	86.92987864
Data Write			20000004H	00000004H	080001A2H	X	6258284606	86.92061953
Exception Exit		15				X	6258284606	86.92061953
Exception Return		0				X	6258284606	86.92061953
Exception Entry		15					6259000288	86.93055956
Data Write	X		20000004H	0000000BH	080001A2H	X	6259009572	86.93068850
Exception Return	X	0				X	6259009572	86.93068850
Exception Entry		15					6259720296	86.94055967
Data Write	X		20000004H	0000000CH	080001A2H	X	6259728338	86.94067136
Exception Return	X	0				X	6259728338	86.94067136
Exception Entry		15					6260440304	86.95055978
Data Write	X		20000004H	0000000DH	080001A2H	X	6260448344	86.95067144

Exception Trace

- ▶ Provides information about exceptions. Real-time
- ▶ Count and times as shown below.



The screenshot shows a window titled "Exception Trace" with a table of exception data. The table has 10 columns: Num, Name, Count, Total Time, Min Time In, Max Time In, Min Time Out, Max Time Out, First Time [s], and Last Time [s]. The data is as follows:

Num	Name	Count	Total Time	Min Time In	Max Time In	Min Time Out	Max Time Out	First Time [s]	Last Time [s]
2	NMI	0	0 s						
3	HardFault	0	0 s						
4	MemManage	0	0 s						
5	BusFault	0	0 s						
6	UsageFault	0	0 s						
11	SVCall	0	0 s						
12	DbgMon	0	0 s						
14	PendSV	0	0 s						
15	SysTick	18324	84.375 ms	0 s	39.980 us	3.293 ms	6.665 ms	0.03109988	61.11085556
16	ExtIRQ 0	0	0 s						
17	ExtIRQ 1	0	0 s						
18	ExtIRQ 2	0	0 s						
19	ExtIRQ 3	0	0 s						
20	ExtIRQ 4	0	0 s						
21	ExtIRQ 5	0	0 s						
22	ExtIRQ 6	0	0 s						
23	ExtIRQ 7	0	0 s						

Example 1

- ▶ Here only Data writes displayed in Trace Records.
- ▶ Write to address 0x20000004 – see the memory window.
- ▶ Data increments 1.
- ▶ Timestamp shown.
- ▶ Both windows are updated in real time!

Type	Opf	Num	Address	Data	PC	Dly	Cycles	Time[s]
Data Write	X		20000004H	0000005DH	080001A2H	X	6246052523	86.75072949
Data Write	X		20000004H	0000005EH	080001A2H	X	6246772529	86.76072957
Data Write	X		20000004H	0000005FH	080001A2H	X	6247492535	86.77072965
Data Write	X		20000004H	00000060H	080001A2H	X	6248212541	86.78072974
Data Write	X		20000004H	00000061H	080001A2H	X	6248932547	86.79072982
Data Write	X		20000004H	00000062H	080001A2H	X	6249652553	86.80072990
Data Write	X		20000004H	00000063H	080001A2H	X	6250372559	86.81072999
Data Write	X		20000004H	00000000H	080001AAH	X	6251117985	86.82108313
Data Write	X		20000004H	00000001H	080001A2H	X	6251812571	86.83073015
Data Write	X		20000004H	00000002H	080001A2H	X	6252532577	86.84073024
Data Write	X		20000004H	00000003H	080001A2H	X	6253252583	86.85073032
Data Write	X		20000004H	00000004H	080001A2H	X	6253972589	86.86073040
Data Write	X		20000004H	00000005H	080001A2H	X	6254692595	86.87073049
Data Write	X		20000004H	00000006H	080001A2H	X	6255412601	86.88073057
Data Write	X		20000004H	00000007H	080001A2H	X	6256132607	86.89073065
Data Write	X		20000004H	00000008H	080001A2H	X	6256852613	86.90073074
Data Write	X		20000004H	00000009H	080001A2H	X	6257572619	86.91073082

Serial-Wire ... Logic Anal... Serial.c

Address: 0x20000004

0x20000004:	09	00	00	00	00	00	00	00	00	00	00	00	00	4E	02	00	FF	03	10	00	00	4F	02	00
0x20000001F:	00	01	01	00	00	00	0E	00	00	00	00	00	00	01	00	00	00	4C	24	01	40	10	00	00
0x20000003A:	00	00	01	00	00	00	00	00	00	00	00	00	00	00	00	00	01	00	00	00	04	00	00	20
0x200000055:	20	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x200000070:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	01	00	00	00	00	00	00	00

Example 2

- ▶ Cortex-M3 evaluation board. 125 mHz.
- ▶ Program is caught in loop 1922 to 1926.
- ▶ Trace shows you quickly where you have been.
- ▶ PC Sample good for Profile Analysis.
- ▶ Note we missed many instructions in this fast tight loop.
- ▶ If we want all PCs – must use ETM.

Trace Records								
Type	Ovf	Num	Address	Data	PC	Dly	Cycles	Time[s]
PC Sample					00001924H		13300456799	601.51952382
PC Sample					00001922H		13300473183	601.51985150
PC Sample					00001922H		13300489567	601.52017918
PC Sample					00001922H		13300505951	601.52050686
PC Sample					00001926H		13300522335	601.52083454
PC Sample					00001926H		13300538719	601.52116222
PC Sample					00001924H		13300555103	601.52148990
PC Sample					00001922H		13300571487	601.52181758
PC Sample					00001922H		13300587871	601.52214526
PC Sample					00001926H		13300604255	601.52247294
PC Sample					00001926H		13300620639	601.52280062
PC Sample					00001924H		13300637023	601.52312830
PC Sample					00001922H		13300653407	601.52345598
PC Sample					00001926H		13300669791	601.52378366
PC Sample					00001926H		13300686175	601.52411134
PC Sample					00001924H		13300702559	601.52443902
PC Sample					00001922H		13300718943	601.52476670
PC Sample					00001922H		13300735327	601.52509438
PC Sample					00001926H		13300751711	601.52542206
PC Sample					00001926H		13300768095	601.52574974

```
565: //  
0x00001922 680A LDR r2,[r1,#0x00]  
566: if(u1X == 128)  
567: {  
568:     return;  
569: }  
570: }  
571:  
572: //  
573: // Advance to the next character.  
574: //  
0x00001924 2A00 CMP r2,#0x00  
0x00001926 D0FC BEQ 0x00001922  
537: for(u1Idx1 = 0; u1Idx1 < 6; u1Idx1 += 2)  
0x00001928 600B STR r3,[r1,#0x00]
```

ARM

Serial Wire Viewer SWV can see:

- ▶ Global and Static variables, Structures.
- ▶ Peripheral registers – just read or write to them.
- ▶ Can't see local variables. (just make them global or static)
- ▶ Can't see DMA transfers – DMA bypasses CPU.
- ▶ SWV is very easy to setup and use.
- ▶ SWV doesn't have ETM's throughput – but is still powerful.
- ▶ And no expensive ETM adapters are needed – ULINK2 or ME, ULINKPro, ST-Link or Segger J-Link !

Output Data of SWV and ETM - differences

- ▶ Serial Wire Viewer
 - Program Counter (PC) samples (from every 64th to 5,000)
 - Exceptions – both system and external (to core).
 - Data read and writes.
 - ITM – Instruction Trace Macrocell. **Printf** outputs.
 - Some counters on CPU operations like folded instructions etc.
 - Timestamps.
- ▶ ETM
 - Every PC is captured and timestamped 😊
- ▶ **Note:** you can see SWV and ETM go hand in hand !

DCC and ITM – what are these ?

- ▶ A method to send or receive messages from user to debugger.
- ▶ Except for your read or write to a 32 bit register – is real-time.
- ▶ Data Communications Channel: DCC
 - send and receive – 1 transmit and 1 receive register.
 - ARM EmbeddedICE only.
 - ARM has “Real Time Agent” to send info out DCC. Intrusive.
- ▶ Instruction Trace Macrocell: ITM
 - Transmit only. CoreSight only. Replaces DCC.
 - 32 32-bit Stimulus Registers.
 - This is the printf debugging.
- ▶ Can R/W memory real-time with Memory Access Port.

How to use the ITM:

- ▶ Define it:
 - ▶ `#define ITM_Port8(n)((volatile unsigned char*)(0xE0000000+4*n))`
- ▶ Send some data out the SWO pin:
 - `ITM_Port8(0) = ch;`
- ▶ The write to ITM intrusive – all other operations to get this out the SWO pin are non-intrusive.
- ▶ This is much easier to use than Real Time Agent with DCC.

Summary:

- ▶ Three ways to connect to the debugger in ARM microcontrollers:

✓ Note: To Start, Stop, set breakpoints & watchpoints, R/W memory.

- 1) Regular JTAG connector.
- 2) ETM connector.
- 3) Serial Wire Debug (SW-DB) (this shares the JTAG connector).

- ▶ Five ways to get debug information out of an ARM processor:

- 1) Real Time Agent: Debug Communications Channel (DCC) - JTAG (not CorSight)
- 2) ETM: Embedded Trace Macrocell. (not all processors have this option)
- 3) Serial Wire Viewer (SWV): out the Serial Wire Output pin (SWO).
- 4) ITM Viewer – Instruction Trace Macrocell (comes out SWO pin).
- 5) Plus, well OK sure – stop the processor. But you will stop it a lot.

PCB Hints for Cortex-M3 ETM™

- ▶ 1. Only one trace connector (preferably new 20-pin connector).
- ▶ 2. Physical placement (as close to the chip as possible) - it is easy to do so with new, compact 20-pin connector.
- ▶ 3. JTAG connector (new 10-pin) should be there too, but do not need to be so close to the CPU. Please note, that 10 pins on that connector are same as exactly the half of new 20-connector, so all JTAG pins may "pass through" the trace 20-pin connector.
- ▶ 4. How to run TRACECLK and TRACEDATA signals on PCB: short and parallel to each other, best would be a ground in between them. Note that all trace pads are in the middle of one side of the CPU package. There are GND and VCC pads in the middle of trace pads.
- ▶ 5. Orientation for proper clearance and orientation with emulators.
- ▶ 6. Pads for new 20-pin ETM connector should be on every board –the new connector does not take too much space.
- ▶ Remember Cortex-M3 parts gone = 144 mHz so layout more important.

What else is there ?

- ▶ **Watchpoints**
 - Conditional breakpoints – stop program when xyz...
- ▶ **PTM: Program Trace Macrocell**
 - For Cortex-A8 and A15 and beyond.
 - No data tracing. ITM tracing exists.
- ▶ **HSST: High Speed Serial Trace**
 - Exists today.

What is in the future ?

- ▶ **STM: System Trace Macrocell**
 - Up to application level debugging.
 - On Cortex-R and A – will eventually replace ITM.
- ▶ **TMC: Trace Memory Controller**
 - Adds ability to send data out Ethernet or HSSTP.
- ▶ **Linux application aware debuggers using trace.**

Conclusions:

- ▶ Trace is an excellent feature.
- ▶ Now you know how this all fits together.
- ▶ ETM and SWV are easier to use than you might think.
- ▶ Thank you !

Comments:

Robert Boys

Bob.boys@arm.com

San Jose, California

The ARM logo is displayed in a large, bold, blue sans-serif font. A small registered trademark symbol (®) is located to the upper right of the letter 'M'.The Cortex logo features the word 'Cortex' in a blue sans-serif font, with a small trademark symbol (™) to its upper right. Above the 'x' is a decorative graphic of four squares in a 2x2 grid, with the top-left square being blue and the others black. Below the word 'Cortex' is the tagline 'Intelligent Processors by ARM®' in a smaller, black sans-serif font.A smaller version of the ARM logo, consisting of the letters 'ARM' in a bold, blue sans-serif font.